OPTIMIZED DAY AND LOW-LIGHT ROAD DAMAGE DETECTION WITH AUTOMATED RISK ASSESSMENT

A Master's Dissertation

submitted in partial fulfilment of the requirements for the award of the degree of

MSc in Artificial Intelligence

By

AKASH KUMAR
P2943376

Supervisor: Professor Sumaiya

School of Computing
De Montfort University, Dubai

September 2026

DECLARATION

I declare that this dissertation is my own original work carried out under the supervision of Professor Sumaiya and that all sources and references have been acknowledged appropriately. This dissertation has not been submitted previously for academic credit at De Montfort University, Dubai, or any other institution.

Signature: 

Name: Akash Kumar

Date: 12 September 2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my supervisor, Professor Sumaiya, for her guidance, support, and valuable feedback throughout this project. I would also like to thank the academic staff at De Montfort University Dubai for their support during my MSc studies. Finally, I am grateful to my family for their encouragement and support throughout my studies and the completion of this dissertation.

Akash Kumar

LIST OF FIGURES

Figure 3.1 Crack Model - Synthetic Day-to-Night Transformation 10
Figure 3.2 System Architecture 11
Figure 3.3 YOLO26m Internal Architecture (Backbone, Neck, Head) 12
Figure 3.4 Full Two-Phase Training Pipeline (Pothole Model)..... 15
Figure 3.5 Pothole Model - Data Augmentation Pipeline..... 16
Figure 4.1 Logical relationship between the Final 4 pothole workspace, Crack 3 workspace, and deployment components 20
Figure 4.2 Experimental multi-model selection interface used during development..... 23
Figure 4.3 Final dual-model web application showing separate pothole and crack outputs..... 24
Figure 4.4 Live camera implementation showing separate crack and pothole detection windows .. 25
Figure 5.1 mAP50 progression across the main model-development experiments 29
Figure 5.2 Dedicated crack-model progression from Crack-only to Crack 3 30
Figure 5.3 Crack 2 qualitative validation example - ground-truth labels and model predictions 31
Figure 5.4 Final 4 Phase 2 training and validation curves 31
Figure 5.5 Final 4 F1-confidence, precision-confidence, precision-recall and recall-confidence curves 32
Figure 5.6 Final 4 raw confusion matrix..... 33
Figure 5.7 Final 4 performance on day/non-night and night test subsets 34
Figure 5.8 Final 4 pothole annotation distribution and bounding-box geometry..... 35
Figure 5.9 Final 4 qualitative validation example - ground-truth annotations followed by model predictions 36
Figure 5.10 Crack 3 training and validation curves 37
Figure 5.11 Crack 3 F1-confidence, precision-confidence, precision-recall and recall-confidence curves..... 38
Figure 5.12 Crack 3 raw confusion matrix 39
Figure 5.13 Crack 3 annotation distribution and bounding-box geometry 40
Figure 5.14 Crack 3 qualitative validation example - ground-truth annotations and model predictions 41

Figure 5.15 mAP50 before and after FP16 ONNX export 42

LIST OF TABLES

Table 2.1 Summary of Reviewed Literature on Road Damage Detection 7
Table 3.1 Dataset Summary 9
Table 3.2 Final Training Configuration 15
Table 4.1 Development environment used for implementation 19
Table 4.2 Pothole dataset image distribution used by Final 4 21
Table 4.3 Crack 3 image distribution after preprocessing and synthetic low-light generation 22
Table 5.1 Final model evaluation configuration 27
Table 5.2 Experimental model development summary 28
Table 5.3 Final 4 validation performance 31
Table 5.4 Final 4 day/non-night and night test results 33
Table 5.5 Final 4 performance by dataset source 33
Table 5.6 Crack 3 validation performance 37
Table 5.7 Per-class mAP50 in the combined pothole-and-crack models 41
Table 5.8 Original and FP16 ONNX model comparison 42
Table 5.9 Contextual comparison with selected published road-damage studies 44

LIST OF ABBREVIATIONS

Abbreviation	Full Form
YOLO	You Only Look Once
RDD	Road Damage Dataset
mAP	mean Average Precision
F1	F1 Score
GPS	Global Positioning System

ABSTRACT

Road damage such as potholes and cracks presents a persistent safety and maintenance challenge, and existing automated detection systems remain limited in several practical respects: they are rarely tested across both daytime and nighttime conditions together, and they stop at locating damage without indicating its severity. This project sets out to build a road damage detection system, based on the YOLO object detection framework, that addresses these limitations directly. The approach combines daytime imagery from the RDD2022 benchmark dataset with additional nighttime and unpaved gravel-road sources into a single training pipeline, and treats pothole and crack detection as two separate, independently trained models rather than one combined detector, an approach also supported by similar findings in the existing literature. The system further incorporates a hazard scoring mechanism, rated from 1 to 10 based on defect size and type, intended to support maintenance prioritisation. Model size and live webcam performance were also examined to assess how practical the system would be to run on lightweight hardware. The final system demonstrated reliable road-damage detection across varied road and lighting conditions, while the use of separate pothole and crack models, automated risk scoring, model compression, and live-camera testing showed the practical potential of the proposed approach.

Keywords: Road Damage Detection, YOLO, Object Detection, Low-Light Detection, Model Compression, Risk Assessment

TABLE OF CONTENTS

Declaration ii
Acknowledgement iii
List of Figures iv
List of Tables v
List of Abbreviations vi
Abstract vii

Chapter 1: Introduction 1
1.1 Background 1
1.2 Problem Statement 2
1.3 Aim and Objectives 2
1.4 Research Questions 3
1.5 Significance 3
1.6 Scope 3
Chapter 2: Literature Review 4
2.1 Datasets and the Shift Toward YOLO 4
2.2 Making Models Smaller and Faster 4
2.3 Vehicle and Drone-Specific Deployment 5
2.4 Detecting Damage in the Dark 5
2.5 One Model or Two 6
2.6 What This Leaves Unaddressed 6
Chapter 3: Methodology 9
3.1 Overview of Approach 9
3.2 Data Collection and Sources 9
3.3 Data Preparation 10
3.4 Model Architecture 11
3.5 Training Configuration 14
3.6 Hazard Risk-Scoring Method 17
3.7 Model Compression Approach 17
3.8 Web Application Design 17
3.9 Evaluation Metrics 17
Chapter 4: System Implementation 19
4.1 Development Environment and Project Organisation 19
4.1.1 Project Directory Hierarchy 19
4.2 Dataset Processing Implementation 20
4.3 Pothole Model Implementation - Final 4 22
4.4 Crack Model Implementation - Crack 3 22
4.5 Experimental Model Selection Interface 23
4.6 Final Dual-Model Web Application 23
4.7 Automated Risk-Scoring Implementation 24
4.8 Live Camera Detection 25
4.9 Model Compression Implementation 25
4.10 Chapter Summary 25
Chapter 5: Results and Discussion 27
5.1 Chapter Overview 27
5.2 Evaluation Setup and Scope 27
5.3 Model Development and Selection 27
5.3.1 Early YOLO11 Development 29
5.3.2 Pothole and Combined-Model Progression 29
5.3.3 Dedicated Crack Model Progression 29
5.4 Final Pothole Model Results - Final 4 31

5.4.1 Overall Validation Performance 31
5.4.2 Training Behaviour 31
5.4.3 Confidence and Precision-Recall Behaviour 32
5.4.4 Confusion Matrix Analysis 33
5.4.5 Day and Night Test Performance 33
5.4.6 Annotation Distribution and Qualitative Pothole Results 35
5.5 Final Crack Model Results - Crack 3 37
5.5.1 Overall Validation Performance 37
5.5.2 Training Behaviour 37
5.5.3 Confidence and Precision-Recall Behaviour 38
5.5.4 Confusion Matrix Analysis 39
5.5.5 Annotation Distribution and Qualitative Crack Results 40
5.6 Combined Models Compared with Specialist Models 41
5.7 Model Compression Results 42
5.8 Real-World System Evaluation 43
5.8.1 Deployment Confidence and Automated Risk Score 43
5.8.2 Image, Video and Live-Camera Operation 43
5.9 Comparison with Existing Studies 44
5.10 Discussion of the Research Questions 45
5.10.1 Research Question 1 - Effect of Combining Road Conditions 45
5.10.2 Research Question 2 - One Combined Model or Two Specialist Models 45
5.10.3 Research Question 3 - Live Operation on Consumer Hardware 45
5.10.4 Compression and Deployment Finding 45
5.11 Chapter Summary 45
Chapter 6: Conclusion 46
Chapter 7: Future Work 47
7.1 Quantisation and Deployment Optimisation 47
7.2 Unified Detection Output 47
7.3 Model Accuracy and Attention Mechanisms 47
7.4 Data and Risk-Score Validation 47
References 48

CHAPTER 1: INTRODUCTION

1.1 Background

Road surfaces do not stay in good condition forever. Constant traffic, changing weather, and simple wear over time all cause gradual damage, and two of the most common problems that result are potholes and cracks. If left unaddressed, both tend to get worse, and both create a genuine risk for drivers. For many years, the only real way to catch this kind of damage early was to send people out to check the roads directly, driving or walking along a route and noting down what they found. This works, but it is slow, costly to keep doing regularly, and puts inspectors close to moving traffic while they work.

The cost of this kind of damage is not limited to the road itself. Driving repeatedly over potholes and cracks places extra strain on a vehicle's suspension, tyres, and wheel alignment, and this added wear tends to build up gradually rather than showing itself all at once, meaning many drivers absorb the cost without necessarily connecting it back to the state of the road. Fuel economy suffers in a similar way, since a driver constantly slowing down, swerving, or absorbing impacts from damaged road surfaces uses more fuel than one travelling on a smooth, well-maintained road. This becomes an even more pressing concern with the growing use of autonomous and semi-autonomous vehicles, which rely on consistent, predictable road conditions to plan safe routes and maintain efficient driving behaviour; unexpected or poorly documented damage can force these systems into sudden corrections. Seen this way, road damage detection is not just a matter of infrastructure upkeep, but one with a real, ongoing cost to everyday vehicle use and to the vehicles themselves.

A camera paired with a trained detection model can now take on a large part of that same task. Once a camera is mounted on a vehicle's dashboard, it can simply record the road as the vehicle goes about its normal route, and a model can be left to review that footage afterwards rather than requiring a dedicated inspection trip. Among the detection architectures suited to this kind of work, the YOLO family has become a particularly common choice, largely because it processes a whole image in one pass instead of scanning it through several stages, which keeps it fast enough to handle live or near-live video (Abdelwahed *et al.*, 2025) Later versions of YOLO have also become smaller and cheaper to run while still improving in accuracy, so running one of these models on an ordinary laptop rather than a dedicated server is now a genuinely realistic option.

This shift has not reached every road authority equally, however. The equipment traditionally used for automated road surveys, laser scanning rigs, road profilers, and 3D sensors among them, is expensive both to buy and to maintain, with dedicated inspection vehicles fitted with this kind of equipment able to cost millions of dollars depending on the sensors involved (Alkalbani *et al.*, 2026) and that cost places it out of reach for a good number of smaller or lower-budget road authorities and contractors. Genuine real-time processing also remains rare in practice; a recent survey of the field concludes that, despite substantial progress in the underlying deep learning methods, a practically deployable, real-time road damage detection system remains largely unrealised (Abdelwahed *et al.*, 2025) A system built instead around an ordinary camera and consumer-grade hardware avoids most of that cost, which is part of the motivation behind the approach taken in this project.

Two further limitations, beyond cost, shaped the direction of the work that follows. Detection accuracy tends to drop sharply once lighting becomes difficult, whether from nightfall, shadow,

or glare from oncoming headlights (Bučko *et al.*, 2022)), and most published systems are only ever tested under the same conditions they were trained on, meaning a model built for daytime is rarely also tested at night, or the other way around. Separately, very few of these systems assign a risk or severity score to what they detect, and this remains uncommon within the studies reviewed here. This project takes on both problems directly and adds a third consideration that is rarely examined elsewhere: how well detection holds up on unpaved and gravel surfaces, tested alongside genuine live camera use rather than static images alone.

1.2 Problem Statement

Three problems keep coming up across existing road damage detection research. The first problem is that lighting makes a real difference to how well these models work. A model trained mostly on daytime photos often misses real damage, or wrongly flags something that isn't actually there, once it gets dark (Hu *et al.*, 2025) Models built specifically for night-time do exist, but they are almost always trained and tested completely separately from daytime models, so there is rarely one single model that can be trusted to work reliably in both conditions (Ling *et al.*, 2024)

The second problem concerns how little these systems actually communicate. A bounding box tells a maintenance team that something is wrong with the road, but says nothing about how urgently it needs attention, and that missing piece of information is exactly what would be needed to prioritise a limited repair budget sensibly.

The third problem is that most research never really tests whether a model is actually ready to be used in practice. A study will often report good accuracy on a set of test images, but very few studies then check what happens to that accuracy once the model is compressed to run on cheaper hardware, or connected to a real, live camera feed instead of a fixed batch of photos. This matters because a maintenance vehicle is very unlikely to carry expensive, powerful computer equipment, so without this kind of testing, there is no real way to know if a model would actually keep working once deployed.

These three problems do not sit apart from one another. A model built specifically to cope with darkness usually ends up larger and slower as a result, which makes the deployment question harder rather than easier to answer. Addressing one of these issues while ignoring its effect on the other two risks simply trading one weakness for another, rather than genuinely solving anything.

1.3 Aim and Objectives

The aim of this project is to build a road damage detection system that performs reliably across different lighting conditions, and to test how genuinely ready that system is for real-world deployment, both through a hazard scoring mechanism and through actual model compression testing rather than a theoretical discussion of it.

To achieve this, the project sets out to:

- Combine daytime and nighttime road imagery, drawing on RDD2022 (Arya *et al.*, 2024) alongside additional nighttime and off-road sources
- Train and compare YOLO-based models capable of identifying cracks and potholes, using a mix of image qualities, including blurry, medium-quality, and high-resolution (4K) images, across a range of different environments.

- Design a hazard scoring system, rated from 1 to 10, based on the size and type of each detected defect.
- Build a web application capable of processing uploaded images and video, as well as running on a live webcam feed.

1.4 Research Questions

This project works toward answering three specific questions:

- Does combining daytime, nighttime, and off-road imagery into a single training set improve accuracy and reliability compared with training on any one of those conditions alone?
- Does training two separate models, one for potholes and one for cracks, perform better than training a single model to detect both at once?
- Can the resulting system process a live webcam feed at an acceptable speed using ordinary consumer hardware, rather than specialised equipment?

1.5 Significance

A detection system built around nothing more than a standard dashboard camera or webcam could, in principle, be fitted to almost any vehicle already travelling through a city, turning an ordinary daily route into a steady source of road condition data. Catching a small crack before it develops into a larger, more expensive pothole represents a genuine opportunity for cost savings, and a direct improvement in safety for everyday drivers. This project also contributes something less immediately practical but still worth noting: a properly tested comparison between two possible designs, one combined detection model against two separate, specialised ones, a question that has received surprisingly little direct attention in the wider literature (Verma *et al.*, 2025)). A further strength of the approach taken here is that a single system is trained to work across several conditions at once, day, night, and unpaved gravel surfaces, rather than being limited to just one, which is closer to how roads are encountered in practice.

1.6 Scope

Cracks and potholes are handled by two separate, independently trained models rather than a single combined one, a decision explained in the chapter that follows. The technical work covers dataset preparation, model training, and the construction of a web-based interface that can process both uploaded media and a live webcam stream. Evaluation is limited to detection accuracy and processing speed measured in frames per second. Scheduling actual repairs, integrating with existing council or municipal software, and detecting damage types such as rutting or faded road markings, all fall outside what this project attempts to cover. Where the term "off-road" is used, it refers specifically to unpaved or gravel-surfaced roads of the kind found on rural or lower-priority routes, rather than off-road terrain in a recreational or motorsport sense.

CHAPTER 2: LITERATURE REVIEW

2.1 Datasets and the Shift Toward YOLO

Progress in road damage detection depends heavily on a small number of shared datasets, the most significant of which is RDD2022, covering 47,420 images across six countries and four damage categories (Arya *et al.*, 2024) It is worth noting that the strongest model produced from the competition built around this dataset reached 77% accuracy overall, yet fell below 60% on two of the six national subsets, and this kind of gap between a strong headline figure and much weaker performance on individual conditions shows up repeatedly throughout the wider literature. Before datasets of this scale existed, road condition surveys relied on physical sensors, laser rigs and vibration meters among them, and while these work reasonably well, they cost too much for most authorities to use routinely. This cost is a significant part of why the field has shifted so heavily toward ordinary cameras combined with deep learning instead (Abdelwahed *et al.*, 2025)

Within that broader shift, two competing approaches to detector design are worth distinguishing. Two-stage detectors, Faster R-CNN being the clearest example, first propose likely locations for damage and then check each one seperately, an approach that tends to be accurate but slow. One-stage detectors, and YOLO in particular, skip that two-step process entirely and evaluate the whole image in a single pass, trading a small amount of theoretical accuracy for a considerable gain in speed. (Pham, Nguyen and Donan, 2022) illustrate both the strength and the limits of this trade-off clearly: their YOLOv7 model, trained partly on road footage they collected themselves from the United States, reached an F1 score of 82% on that specific US data, but only 74% once tested across all six countries in the dataset, and the same model performed noticeably worse whenever it was applied to a country it had not originally been trained on.

2.2 Making Models Smaller and Faster

A second, closely related strand of research focuses less on raw accuracy and more on making these models small and fast enough to actually deploy. Several groups have replaced YOLO's standard backbone with GhostNet, an approach that produces feature maps more cheaply than a full convolution operation would. (Zhang *et al.*, 2022)) combined GhostNet with a custom attention module and reached 56.9% accuracy on the GRDDC2020 dataset while using considerably fewer parameters than heavier alternatives, although shadows falling across the road surface still tended to trigger false alarms. (Lin, Zhi and Zhang, 2023) took a broadly similar route with a model they called Ghost-CANet, cutting the parameter count by close to half while still reaching 0.66 mAP, and their own testing found something worth remembering: adding an attention mechanism without the Ghost modules that were meant to accompany it actually made accuracy worse, not better, which suggests these techniques cannot simply be stacked together and expected to help independently.

(Wan *et al.*, 2022) achieved a similar balance of size and speed with a model called YOLO-LRDD, though thin, broken transverse cracks remained difficult for it to detect reliably. (Wang *et al.*, 2025) instead focused specifically on small and irregularly shaped damage, adding a wavelet-based filtering step that improved detection of small objects at the cost of a somewhat heavier, slower model. (Khare *et al.*, 2023) compared several sizes of YOLO directly with embedded deployment in mind, and found the smallest, YOLOv8 Nano, to be the clear winner on both speed and file size; however, its accuracy dropped sharply, from over 90% down to

under 60%, once measured against the stricter mAP50-95 standard rather than the more commonly reported mAP50.

(Du *et al.*, 2024) approached the same underlying goal differently again, layering an attention mechanism, targeted data augmentation, careful sample balancing, and knowledge distillation from a larger model on top of a small YOLOv8 baseline. Their own breakdown of each technique's individual contribution is genuinely useful: knowledge distillation alone produced by far the largest single improvement of anything they tried. It is worth noting, though, that their eventual competition placing was decided by processing speed rather than accuracy once a minimum accuracy threshold had been met, and their submission in fact had the lowest accuracy of the five highest-ranked teams, a detail easy to miss if only the final ranking is considered.

2.3 Vehicle and Drone-Specific Deployment

A smaller group of studies looks specifically at the hardware constraints introduced by where a model is meant to run. (Khan *et al.*, 2023) compared YOLOv8 directly against YOLOv5 for use in autonomous vehicles and reported near-perfect accuracy, although this result came from a small dataset of under 1,500 images, leaving open the question of how well it would hold up on a wider, more varied set of roads. (Khan *et al.*, 2024) working instead on drone-based inspection, found EfficientDet D1 to be the strongest option once battery life and onboard processing limits were properly accounted for, since heavier two-stage detectors turned out to be impractical on that kind of hardware, and, much like (Pham, Nguyen and Donan, 2022) also observed that a model trained on one country's roads did not transfer particularly well to another's.

2.4 Detecting Damage in the Dark

Sensitivity to lighting conditions is, by some distance, the most heavily researched of the three problems raised earlier in this report. (Bučko *et al.*, 2022) offer one of the clearest demonstrations of just how large this problem can be: accuracy for both YOLOv3 and Sparse R-CNN fell by close to 90% once conditions turned fully dark, with rain and wet-road glare adding further false detections on top of that.

Several research groups have chosen to correct the image itself before detection even begins, rather than expecting the detector alone to cope with darkness (Hu *et al.*, 2025) built a network that transforms nighttime images into something closer to their daylight equivalent before passing them to a detector, and reached accuracy close to what would be expected under normal daytime conditions, though the approach still fails in extremily low light, below roughly one lux, struggles with direct glare from streetlamps, and the heavy computation involved in this transformation step makes genuine live, real-time use difficult(Bai and Zhang, 2025) followed a comparable path, adding a brightness-enhancement stage ahead of detection and reaching just over 91% mAP; their own testing showed that this enhancement step, rather than any change made to the detector's loss function, was responsible for almost all the improvement(Xu *et al.*, 2025) applied similar reasoning specifically to pothole detection and measured a modest but genuine improvement over a plain YOLOv10 baseline as a result.

Not every study in this space relies on a separate enhancement stage, however. (Ling *et al.*, 2024) instead built a dedicated nighttime pothole dataset and modified YOLOv8's internal architecture directly, although their dataset covers potholes only, leaving crack detection at night completely untested(Tsanaullailla *et al.*, 2024) changed nothing about the underlying

architecture at all, focusing purely on tuning the optimiser, learning rate, and batch size, and still reached close to 90% mAP50, although a stricter accuracy measure applied to the same model told a considerably less flattering story(Priyanka, Praveen Raja and Akhib, 2023) went further still, relying only on classical image adjustments, contrast correction and noise removal among them, ahead of an otherwise unmodified YOLOv7, and still recorded a genuine improvement in accuracy, albeit on a fairly small dataset limited to a single city.

(Zanevych *et al.*, 2024) draw several of these ideas together, comparing a number of YOLO model sizes alongside a custom design built around a feature pyramid and an explainability layer, and report their strongest result reaching 0.72. It is worth flagging, in the interest of accuracy, that their paper's conclusion states a different, higher accuracy figure than the one shown in its own results table, a useful reminder to check exactly which metric any given accuracy claim is actually referring to before treating it as directly comparable to another study's figures.

2.5 One Model or Two

Only a small portion of the existing literature addresses a question central to this project: whether pothole and crack detection are better handled by a single combined model, or by two separate ones trained independently. (Verma *et al.*, 2025) provide the clearest evidence either way. After finding that a single model trained to detect both defect types performed poorly, they moved instead to three separate models, one per defect type, and reported this decision as a direct response to that earlier disappointing result. Their crack detector, evaluated on its own, still only reached 0.49 mAP50, with particularly weak results for transverse cracks specifically, but the underlying decision to separate the models in the first place is directly relevant to the present project, and supports treating pothole and crack detection as two distinct problems rather than folding them into one.

2.6 What This Leaves Unaddressed

Taken as a whole, this body of research leaves several gaps only partly closed. Very little of it attempts to turn a raw detection into a severity score that a maintenance team could actually act on. Daytime and nighttime detection are still, for the most part, treated as separate problems rather than combined within a single training process. Unpaved and gravel roads receive almost no direct testing anywhere in the studies reviewed here, and real deployment readiness more broadly remains largely untested. And the choice between one combined model and several specialised ones has, as far as this review has found, only really been examined in depth by (Verma *et al.*, 2025) This project responds to each of these points directly: by combining day, night, and gravel-surface training data into one pipeline, by building a working severity score rather than stopping at detection alone, by genuinely testing model compression rather than assuming its effect, and by deliberately separating pothole and crack detection into two specialised models, following the same reasoning set out above.

Table 2.1: Summary of Reviewed Literature on Road Damage Detection

Reference	Model / Architecture	Focus Area	Number of Images	Key Findings and Limitations
(Abdelwahed *et al.*, 2025)	YOLO & R-CNN (survey)	Real-time detection survey	N/A (survey of multiple datasets)	Confirms YOLO balances speed and accuracy well; notes class imbalance and lack of adverse-weather data across reviewed datasets.
(Arya *et al.*, 2024)	RDD2022 (dataset)	Multi-national benchmark dataset	47,420	47,420 images, 6 countries; winning CRDDC'2022 model reached 77.0% F1 overall but only 58.3-59.5% on India/Norway subsets.
(Bai and Zhang, 2025)	RF-YOLO-RASFF	End-to-end nighttime detection	~3,000 real + darkened RDD2020/2022	Retinexformer + adaptive feature fusion reached 91.27% mAP; enhancement stage drove most of the gain, not the SIoU loss alone.
(Bučko *et al.*, 2022)	YOLOv3 & Sparse R-CNN	Challenging weather/lighting	~2,099	Accuracy dropped ~90% in total darkness for both architectures; rain/puddles caused false positives.
(Du *et al.*, 2024)	Optimized YOLOv8n	Speed-optimized detection (ORDDC'2024)	RDD2022 (47,420)	F1=0.7013, lowest of top-5 ranked teams once speed-based Phase 2 ranking is accounted for.
(Hu *et al.*, 2025)	IllumiShiftNet + YOLOv11	Nighttime image transformation	5,189 (4,660 paired + 529 real night)	Outperformed Pix2Pix/BBDM; fails under 1 lux and with direct streetlamp glare.
(Khan *et al.*, 2023)	YOLOv8 vs YOLOv5	Autonomous vehicles	1,497 (665 base, augmented)	Near-perfect accuracy on a small (665-1,497 image) augmented Kaggle dataset; braking-system integration untested.
(Khan *et al.*, 2024)	EfficientDet D1 (TinyML)	UAV/drone deployment	8,476	~86.5% accuracy on 8,476-image RDD2022 subset; two-stage models too heavy for drones; poor cross-country generalisation.
(Khare *et al.*, 2023)	YOLOv8 Nano/Small vs v5/v7	Embedded hazard detection	10,613 (3,770 base, augmented)	YOLOv8n: 91.1% mAP50 at 8.8ms/image, but only 59% at stricter mAP50-95.
(Zhang *et al.*, 2022)	GhostNet + LMCA-Net	Lightweight edge deployment	21,000+ (GRDDC2020)	56.9% accuracy on GRDDC2020 with far fewer parameters; false positives under heavy shadow.
(Lin, Zhi and Zhang, 2023)	Ghost-CANet (YOLOv5s)	Lightweight edge deployment	42,824-annotation RDD2022 + 438 new	46.94% parameter reduction, 0.66 mAP@0.5; Coordinate Attention alone (no Ghost modules) reduced accuracy.
(Ling *et al.*, 2024)	WT-YOLOv8	Nighttime pothole benchmark (NPD)	3,800+	0.585 mAP on new 3,800+ image nighttime dataset; pothole-only, weak on unlit rural roads.

(Pham, Nguyen and Donan, 2022)	YOLOv7 + coordinate attention	Multi-country generalisation	RDD2022 + 6,005 self-collected	81.7% F1 (US), 74.1% (all 6 countries); models trained on one country generalised poorly to others.
(Priyanka, Praveen Raja and Akhib, 2023)	YOLOv7 + preprocessing	Smartphone nighttime detection	1,170	0.672 -> 0.764 mAP after contrast/unsharp/median-filter preprocessing; small, geographically narrow dataset.
(Tsanaullailla *et al.*, 2024)	YOLOv8 (hyperparameter study)	Nighttime pothole monitoring	3,685	Best config (Adam, batch 16): 89.9% mAP50 but only 54.6% mAP50-95.
(Verma *et al.*, 2025)	3 independent models (crack/pothole/waterlogging)	Multi-model decoupled detection	2,559 (2,047 train + 512 val)	Crack-only mAP50=0.49; explicitly states a combined single model was unsatisfactory, motivating decoupled design.
(Wan *et al.*, 2022)	YOLO-LRDD (Shuffle-ECANet)	Lightweight edge deployment	RDD2020 + 438 new (RDDC)	~29% smaller, 22% faster than YOLOv5s; struggles with thin, discontinuous transverse cracks.
(Wang *et al.*, 2025)	YOLO-RD (Star Operation + Wavelet)	Small/irregular crack detection	7,900	Outperformed YOLOv8s on small objects; heavier/slower; misses faint micro-cracks.
(Xu *et al.*, 2025)	DCE-Pothole-YOLOv10	Low-light enhancement	3,831 (NPD) + custom set	82.8% accuracy, +3.6% over baseline YOLOv10; higher compute cost; untested in snow.
(Zanevych *et al.*, 2024)	YOLOv11 + FPN + Grad-CAM	Low-light pothole comparison study	6,613 (2,188 base, augmented)	mAP50-95=0.72 vs YOLOv8-Medium's 0.611; paper's conclusion reports different (80%/85%) figures than its main table.

CHAPTER 3: METHODOLOGY

3.1 Overview of Approach

Two separate YOLO26m models were trained, one dedicated to pothole detection and one to crack detection, rather than a single combined detector. This decision was reached in two stages: first, it was informed by (Verma *et al.*, 2025) finding that a single combined model performed poorly on both defect types together; second, it was confirmed directly during this project's own testing, where combined two-class models consistently produced a noticeably weaker crack result than an isolated crack-only model trained on the same underlying data. Following the datasets used, how that data was prepared and combined, the model architecture and why it was chosen, the training process for each model, the hazard risk-scoring method, the approach taken to model compression, and the design of the accompanying web application.

3.2 Data Collection and Sources

Table 3.1 summarises every dataset used across the two models. For pothole detection, five sources were combined: two general daytime road-image sets (referred to here as day_v1 and smartathon), a dedicated night-time dataset, HRP4K, and a gravel/unpaved-road dataset. For crack detection, imagery from the Japan and United States subsets of (Arya *et al.*, 2024) was used, filtered down to keep only crack annotations, discarding pothole and non-damage categories present in the original RDD2022 labelling scheme.

Table 3.1: Dataset Summary

Dataset	Used For	Condition	Images
day_v1	Pothole	Day	2,771
night_time	Pothole	Night	3,685
smartathon	Pothole	Day	9,240
HRP4K	Pothole	Day	6,003
gravel	Pothole	Day, off-road/gravel	7,074
RDD2022 (Japan)	Crack	Day	13,133
RDD2022 (United States)	Crack	Day	6,005

Two points about this data selection are worth stating plainly. First, the gravel dataset was included specifically to address the near-total absence of unpaved surfaces. Second, the HRP4K dataset was deliberately included to make the model more robust. While this dataset features highly detailed 4K resolution images, it also contains a natural mix of blurry and lower-quality shots. Rather than filtering the blurry images out, the entire dataset was kept. Training the neural network on a realistic mix of both crystal-clear and imperfect images

forces it to generalise better, teaching it to detect potholes reliably even in the unpredictable, often blurry conditions of real-world dashboard cameras.

3.3 Data Preparation

Raw data arrived in three different annotation formats depending on source: pre-converted YOLO-format text labels (day_v1, smartathon, HRP4K, gravel), and Pascal VOC XML labels (the RDD2022-derived crack sources). A conversion step was written specifically to handle the XML sources, reading each file's bounding-box coordinates and damage-type code, keeping only the codes corresponding to genuine cracks (longitudinal, transverse, and alligator), and converting the remaining boxes into the same normalised YOLO text format used by the other sources. This ensured every dataset, regardless of its original format, was represented identically before training began.

Rather than physically copying or merging image files from different folders into one location, a lightweight approach was used throughout: a script scans each dataset's folder structure automatically, records the file path of every image alongside whether it contains a genuine annotation, and writes three plain text files, one listing every training image path, one for validation, and one for testing. This means the original datasets are never duplicated or altered, and a new dataset can be added to the training set in future simply by placing it in the expected folder structure, without any code changes.

During this process, a small number of corrupted or mismatched label files were identified and excluded automatically, most commonly images whose accompanying label file mixed segmentation-polygon and bounding-box annotation formats within the same file, a formatting inconsistency present in a small subset of the gravel dataset specifically. These images were logged and skipped rather than allowed to silently break the training pipeline.

For the crack model specifically, one additional preparation step was applied: since none of the available crack sources included genuine night-time photography, a synthetic night-time augmentation step was built. This works by taking a copy of each training image and applying three transformations in sequence: an overall brightness reduction, a gamma-correction curve that darkens shadow regions more aggressively than highlights, and a slight shift toward cooler, bluer tones, intended to approximate the general appearance of a road at night more closely than a simple brightness reduction alone would. Since darkening an image does not change where a crack physically sits within the frame, the original bounding-box labels were copied across unchanged to each darkened image. This effectively doubled the crack model's daytime training data with a synthetic night-time counterpart. This approach is acknowledged here as a simplified stand-in for genuine night-time photography, since it does not reproduce real effects such as headlight glare, streetlight illumination, or camera sensor noise under low light.

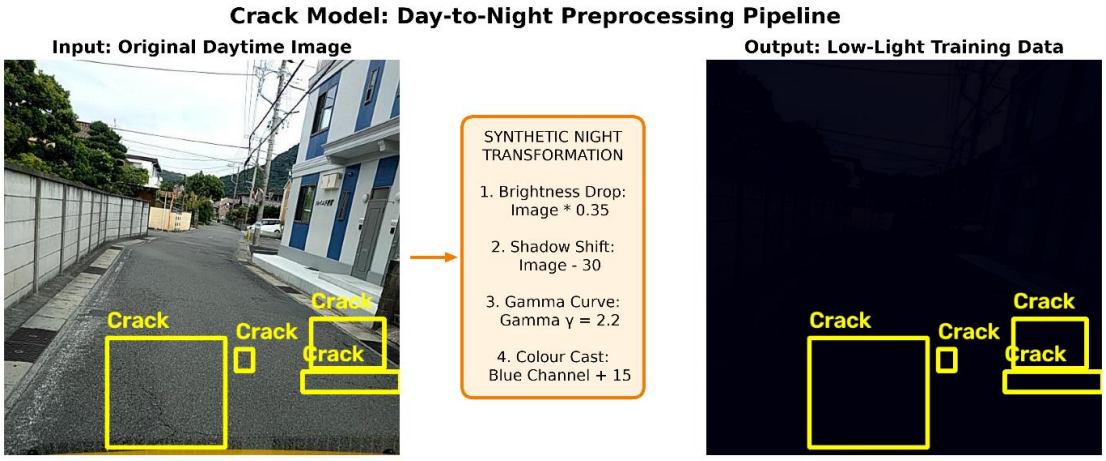


Figure 3.1: Crack Model - Synthetic Day-to-Night Transformation

Figure 3.1 shows this transformation applied to a genuine training image from the Japan subset of RDD2022, with its 4 original crack annotations visible as yellow boxes on both the input and output image. The same four numerical steps described above are shown applied in sequence: the image is first darkened by scaling every pixel value by 0.35 and subtracting 30, then passed through a gamma-correction curve of 2.2 that pushes the darker shadow regions down further than the brighter areas and finally given a slight blue tint by adding 15 to the blue colour channel. Because this process only changes pixel brightness and colour, and never moves or resizes anything within the frame, the same 4 crack bounding boxes remain valid and are copied across unchanged to the darkened output image, visible in yellow on the right.

3.4 Model Architecture

Both the pothole and crack models use YOLO26m, a medium-sized YOLO model chosen to balance detection performance and processing speed. Smaller YOLO models are faster but may reduce detection performance, while larger models require more GPU memory and training time. The standard YOLO26m architecture was used without custom changes, as it already includes a built-in C2PSA partial self-attention layer to help the model focus on important image features

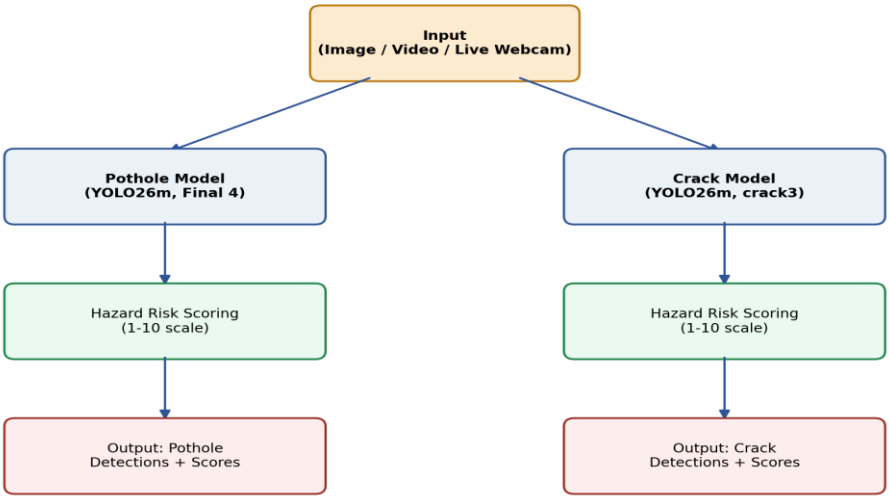


Figure 3.2: System Architecture

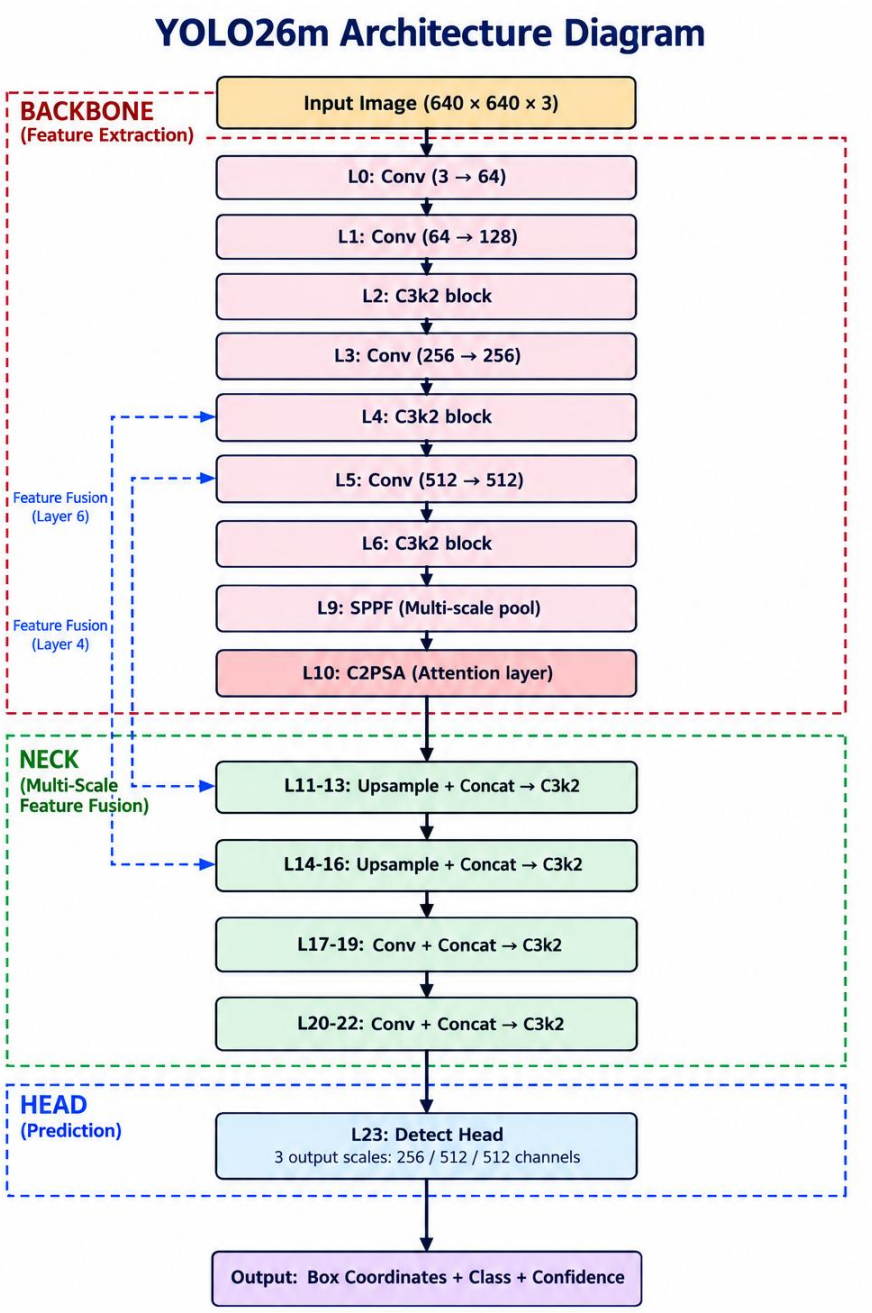


Figure 3.3: YOLO26m Internal Architecture (Backbone, Neck, Head)

The Backbone's job is to extract useful features from the raw image. When a 640 × 640 picture first enters the network, it only has 3 channels (Red, Green, and Blue). In the very first step, L0: Conv (3 → 64), the model expands these 3 colour channels into 64 distinct feature maps. At this early stage, the network is just looking for basic visual shapes, like straight lines and sharp edges.

As the image moves deeper into the network (Layers 1 through 5), the model gradually shrinks the physical width and height of the image but heavily increases the channel depth moving from 64 to 128, then 256, and finally up to 512 channels. By compressing the image's physical

dimensions and stacking more channels, the network stops looking at simple pixels and starts understanding complex semantic patterns, such as the rough texture of alligator cracking or the shadowed rim of a pothole.

To do this efficiently, the network uses C3k2 bottleneck blocks (Layers 2, 4, 6, and 8) to learn these textures without overloading the system's memory. Finally, an SPPF layer (Layer 9) pools information at multiple scales so the model can recognise damage regardless of how far the camera is from the road, and an attention layer (C2PSA, Layer 10) helps the network focus purely on road defects while ignoring background noise like the sky or passing cars.

The Neck and Feature Fusion (Layers 11–22)

By the time the image reaches the end of the Backbone, the network has a deep understanding of what it is looking at. However, because the image has been shrunk down so much, it has lost track of exactly where the sharp boundaries of the damage are. The Neck solves this using a Feature Pyramid Network (FPN) design.

To get those sharp spatial details back, the Neck reaches back into the middle of the Backbone and pulls high-resolution data directly from Layer 6 and Layer 4 using "skip connections" (shown as blue dashed arrows). Because the deep feature maps coming from Layer 10 are too physically small to mix directly with these earlier layers, the Neck first applies an Upsample operation to mathematically double their width and height. It then Concatenates or stacks this expanded data together with the crisp details from Layer 6. It repeats this upsampling and stacking process again for Layer 4.

This fusion step is crucial for road damage detection. It allows the model to successfully combine the deep "big picture" context (knowing it is looking at a damaged road) with the high-resolution boundaries from earlier layers (knowing exactly where the jagged edges of a crack are).

The Head (Prediction, Layer 23)

The final stage is the Detect Head (Layer 23). Because road damage comes in all shapes and sizes from tiny, hairline transverse cracks to massive, deep potholes the network does not rely on just one prediction scale. Instead, it outputs predictions across three different scales: 256, 512, and 512 channels.

- The 256-channel output looks at the image at a higher spatial resolution, making it highly sensitive to detecting small, narrow objects like surface cracks.
- The two 512-channel outputs operate at lower resolutions but with deeper context, which is perfect for drawing accurate bounding boxes around large, complex objects like major potholes.

Together, these outputs produce the final bounding box coordinates, the classification label, and the model's confidence score for each detection

3.5 Training Configuration

The pothole model was trained in two phases, as shown in Figure 3.4. In the first phase, the Backbone was frozen for 15 epochs, while only the Neck and Head were trained using a learning rate of 0.001. This allowed the detection layers to adapt to the pothole task without changing the pretrained features too early. In the second phase, the entire YOLO26m model was unfrozen and fine-tuned for another 85 epochs using a lower learning rate of 0.0001. The Phase 1 checkpoint was used as the starting point, so training continued from the learned weights instead of starting again. This two-stage process helped make the fine-tuning more stable and reduced the risk of damaging useful pretrained features. The crack model used single-phase training from pretrained weights, since there was no previous crack-specific checkpoint to continue from

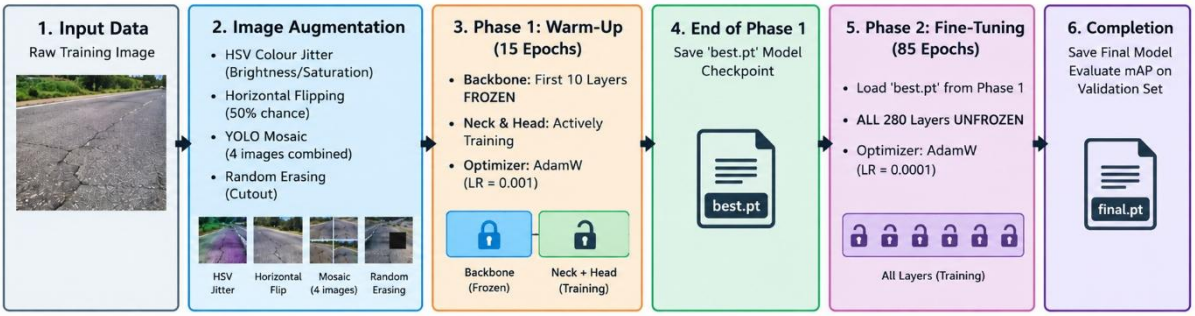


Figure 3.4: Full Two-Phase Training Pipeline (Pothole Model)

Both models were trained using the AdamW optimiser with a deliberately low, flat learning rate, a setting arrived at after testing across multiple earlier models in this project and found to produce more stable convergence than the framework's default learning-rate schedule for this specific combination of dataset size and model size. Table 3.2 summarises the final training configuration used for each model.

Table 3.2: Final Training Configuration

Parameter	Pothole Model (Final 4)	Crack Model (crack3)
Architecture	YOLO26m	YOLO26m
Training approach	Two-phase (frozen backbone, then full fine-tune)	Single-phase
Optimiser	AdamW	AdamW
Learning rate	0.0001	0.0001
Batch size	24	16
Image size	640	800
Epochs (Phase 2 / total)	85	up to 100 (early-stopped)
Patience	15	15

Standard image augmentation was applied during training, including horizontal flipping, moderate hue/saturation/brightness jitter, and mosaic augmentation, which combines four training images into one during early training to improve the model's exposure to varied object scales and positions. Training was allowed to stop automatically if validation accuracy failed to improve for a set number of consecutive epochs, rather than running for a fixed epoch count regardless of whether further improvement was occurring.

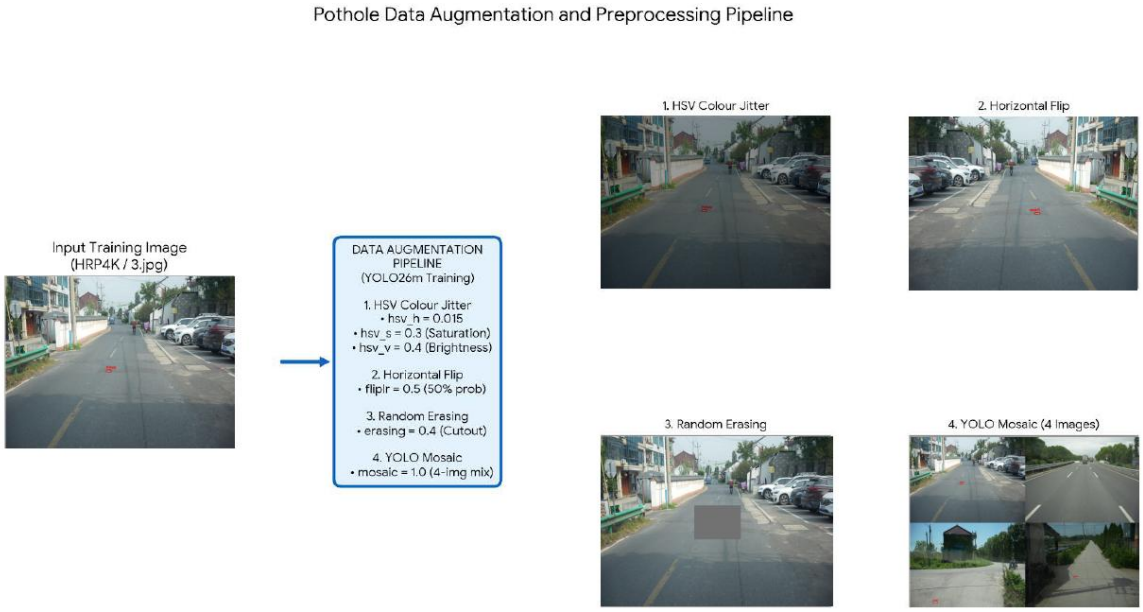


Figure 3.5: Pothole Model - Data Augmentation Pipeline

Figure 3.5 demonstrates each of these augmentations applied to a genuine training image from the HRP4K pothole dataset, using the exact parameter values applied during training. HSV colour jitter shifts the image's hue, saturation, and brightness within a moderate random range (hsv_h=0.015, hsv_s=0.3, hsv_v=0.4), visible here as a subtle overall colour and lighting shift from the original. Horizontal flipping mirrors the image left-right with a 50 percent probability on any given training step; the pothole, marked by the small red box near the centre of the road, remains equally valid reflected. Random erasing blocks out a rectangular patch of the image entirely, shown here as a solid grey square, forcing the model to still recognise surrounding context even when part of the scene is hidden, which helps it cope with real-world occlusions such as another vehicle or shadow partially covering a defect. Mosaic augmentation combines four separate training images into a single composite frame, exposing the network to several different scenes, and several different object scales and positions, within one training step.

3.6 Hazard Risk-Scoring Method

Each detected defect is assigned a hazard score on a 1-to-10 scale, calculated from two factors: the proportion of the image frame occupied by the defect's bounding box, used as a proxy for the physical size of the defect relative to the camera's distance from it, and the defect's class, with potholes weighted more heavily than cracks on the basis that a pothole typically presents a more immediate vehicle-safety hazard than a surface crack of comparable size. The resulting score is combined with the model's own detection confidence to produce a final value, which is then mapped to a colour-coded scale, green for low risk, yellow for medium, and red for high, when displayed to a user. This scoring method is deliberately simple and rule-based rather

than learned from data, since no existing dataset used in this project includes ground-truth severity labels; it is intended as a practical, interpretable first step toward the severity-assessment gap rather than a claim of clinically validated accuracy.

3.7 Model Compression Approach

Post-training reduced-precision export was used to examine whether the final detectors could be made smaller without materially changing their detection behaviour. The Final 4 pothole model was first used for compression experiments, including dynamic and static INT8 trials and an FP16 ONNX export. The more aggressive INT8 variants were treated as experimental candidates, while the FP16 route was retained for reproducible re-validation. After Crack 3 had been selected as the final crack detector, the same FP16 ONNX export procedure was also applied to that model. In every case, the compressed model was evaluated against the same data and object-detection metrics as its original PyTorch checkpoint so that the effect of reduced precision could be measured directly.

3.8 Web Application Design

The completed system is exposed through a web application built with Python's Flask framework. Rather than merging the two models' outputs into a single combined image, the application deliberately keeps the pothole and crack models' results visually separate: an uploaded image or video is processed independently by both models, and two separate annotated outputs are returned side by side, each labelled with its own detection counts and hazard scores. This design choice mirrors the two-model architecture described in Section 3.1 and avoids the visual ambiguity that could arise from overlapping bounding boxes from two different models on a single shared image. The same application also supports a live webcam feed, processing frames from a connected camera in real time using the same dual-model approach.

3.9 Evaluation Metrics

Model performance is assessed throughout this project using four standard object-detection metrics: precision, the proportion of predicted defects that were genuinely correct; recall, the proportion of real defects that were successfully found; mean Average Precision at a 50 percent overlap threshold (mAP50), a commonly reported headline accuracy figure; and mean Average Precision averaged across a stricter range of overlap thresholds from 50 to 95 percent (mAP50-95), a more demanding measure of how precisely each bounding box is positioned. As discussed in Chapter 2, these metrics are not always interchangeable with the accuracy or F1 figures reported elsewhere in the literature, and this project reports all four consistently for every model, rather than selectively citing whichever figure appears most favourable. Inference speed, measured in frames per second, is additionally reported wherever deployment readiness, including the effect of model compression, is being assessed.

CHAPTER 4: SYSTEM IMPLEMENTATION

4.1 Development Environment and Project Organisation

The implementation was carried out in Python using Jupyter Notebook for model development and training, and Flask and OpenCV for application deployment. The final training environment used Python 3.12.10, Ultralytics 8.4.135 and PyTorch 2.11.0 with CUDA 12.8 support. Training was performed on an NVIDIA GeForce RTX 5080 Laptop GPU This environment provided hardware-accelerated training and inference for both YOLO26m models.

Table 4.1: Development environment used for implementation

Component	Implementation
Programming language	Python 3.12.10
Deep-learning framework	PyTorch 2.11.0 + CUDA 12.8
Object-detection framework	Ultralytics 8.4.135 / YOLO26m
Model-development environment	Jupyter Notebook
Image and video processing	OpenCV and NumPy
Web application	Flask with HTML/CSS/JavaScript templates
Training hardware	NVIDIA GeForce RTX 5080 Laptop GPU (~16 GB VRAM)

The implementation was organised into two separate development workspaces: Final 4 for pothole detection and Crack 3 for crack detection. The workspace roots and the main folders and files used during implementation are shown below. Training-run subfolders and detailed output files are intentionally omitted to keep the hierarchy concise and suitable for the thesis.

4.1.1 Project Directory Hierarchy

```
Pothole workspace root: C:\Users\akaka\Desktop\final 2\final 3\final 4\
```

```
final 4/
|
+-- combined_lists/
+-- datasets/
|   +-- Pothole.v1i.yolo26/
|   +-- New pothole detection.v2i.yolo26/
|   +-- night_time/
|   +-- HRP4K/
|   +-- gravel/
|
+-- weights/
+-- static/
+-- templates/
+-- app.py
+-- dual_detect_app.py
+-- dual_live_detect.py
+-- final4.ipynb
+-- Day_To_Night_Pipeline.png
+-- Figure_3_4_Pothole_Augmentation.png
+-- Synthetic_Night_Flowchart.png
+-- yolo26m.pt
```

```
Crack workspace root: C:\Users\akaka\Desktop\final 2\final 3\final
4\final5\final6\final7\crack\crack3\
```

```
crack3/
|
+-- combined_lists/
+-- dataset/
+-- datasets_processed/
+-- Japan_annotated/
+-- United_States_annotated/
+-- weights/
+-- crack3.ipynb
+-- crack3_qc.py
+-- diagnose_crack3.py
+-- yolo26m.pt
```

The directory trees above show the main datasets, notebooks, model folders and deployment files used in the final implementation. Detailed training-run directories, checkpoints and metric outputs are excluded here because they are discussed separately in the training and results sections.

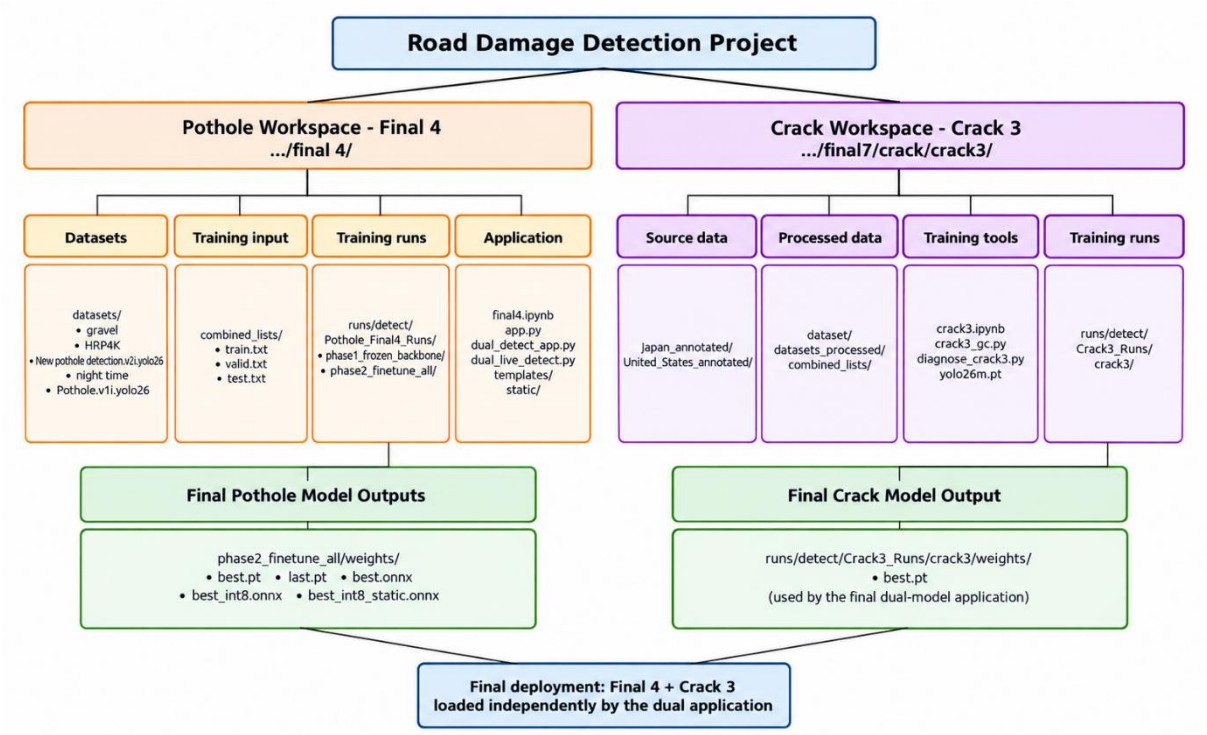


Figure 4.1: Logical relationship between the Final 4 pothole workspace, Crack 3 workspace, and deployment components

4.2 Dataset Processing Implementation

For the pothole detector, the five source datasets remained in their own folders. A notebook routine automatically scanned each dataset, identified the train, validation and test image folders, and generated text manifests containing absolute image paths. The generated train.txt, valid.txt and test.txt files therefore acted as a common input list for YOLO while avoiding the need to copy all source images into a single physical dataset folder. The exact image distribution detected by final4.ipynb is summarised in Table 4.2.

Table 4.2: Pothole dataset image distribution used by Final 4

Dataset	Train	Validation	Test	Total
gravel	5,067	1,345	662	7,074
HRP4K	4,203	900	900	6,003
New pothole detection / Smartathon	6,091	2,094	1,055	9,240
night_time	2,571	742	372	3,685
Pothole.v1i.yolo26	1,939	555	277	2,771
Total	19,871	5,636	3,266	28,773

Across the five pothole sources, 28,773 images were referenced without physically merging the datasets. The Final 4 manifests contained 19,871 training images, 5,636 validation images and 3,266 test images. The class-verification stage also checked 57,084 retained annotation rows and confirmed that every retained bounding box used class 0 (pothole).

Listing 4.1: Path-manifest generation used by the pothole pipeline

```
def write_list(split_name, out_filename):
    all_paths = []
    for ds_name in DATASETS:
        pos, neg = scan_results[ds_name][split_name]
        all_paths.extend(pos)
        all_paths.extend(neg)
    with open(out_path, "w", encoding="utf-8") as f:
        f.write("\n".join(all_paths))
```

The crack pipeline required an additional conversion stage because the Japan and United States RDD2022 subsets used Pascal VOC XML annotations. The implementation retained only D00, D10 and D20 annotations, corresponding to longitudinal, transverse and alligator cracking, and converted each retained bounding box to normalised YOLO format as class 0 (Crack). The processed Japan and United States datasets were stored as separate derived folders, leaving the original source data unchanged.

Listing 4.2: Crack-class filtering and XML-to-YOLO conversion

```
CRACK_CODES = {"D00", "D10", "D20"}

if name not in CRACK_CODES:
    continue

x_center = ((xmin + xmax) / 2) / img_width
y_center = ((ymin + ymax) / 2) / img_height
width  = (xmax - xmin) / img_width
height = (ymax - ymin) / img_height
yolo_lines.append(f"0 {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}")
```

Synthetic low-light copies were generated only from the crack training images. Each original training image was darkened, gamma corrected and shifted slightly toward the blue channel, after which its label file was copied unchanged because no geometric transformation was applied. Validation and test images were not synthetically modified. The resulting image counts used by Crack 3 are summarised in Table 4.3.

Table 4.3: Crack 3 image distribution after preprocessing and synthetic low-light generation

Dataset	Train	Validation	Test	Total
Japan crack-only	9,456	1,050	2,627	13,133
United States crack-only	4,325	480	1,200	6,005
Synthetic low-light crack	13,781	0	0	13,781
Combined Crack 3 input	27,562	1,530	3,827	32,919

The two original crack training subsets contained 13,781 images in total (9,456 from Japan and 4,325 from the United States). Creating one synthetic low-light counterpart for each of these images added a further 13,781 training samples, increasing the Crack 3 training manifest to 27,562 images while leaving the 1,530 validation and 3,827 test images unchanged. Class verification checked 47,655 annotation rows across the processed crack data and confirmed that every retained annotation was mapped to class 0 (Crack).

4.3 Pothole Model Implementation - Final 4

The final pothole detector was implemented as a two-phase fine-tuning procedure using pretrained YOLO26m weights. In Phase 1, the first ten model modules were frozen, and the remaining trainable modules were adapted for 15 epochs at 640 x 640 input resolution. AdamW was used with a learning rate of 0.001, batch size 24 and disk caching. The best checkpoint produced by this phase was then used as the starting point for Phase 2.

In Phase 2, no freeze parameter was supplied, allowing the complete model to be updated. Fine-tuning continued for a maximum of 85 additional epochs using a lower learning rate of 0.0001 and early stopping with a patience of 15 epochs. The final checkpoint was stored under runs/detect/Pothole_Final4_Runs/phase2_finetune_all/weights/best.pt and became the pothole model used by the final application. This implementation therefore preserved the staged procedure described in Chapter 3 while keeping a separate checkpoint for each training phase.

Listing 4.3: Simplified Final 4 two-phase training implementation

```
model_phase1 = YOLO("yolo26m.pt")
model_phase1.train(data=data_yaml_path, epochs=15, imgsz=640,
                   batch=24, freeze=10, optimizer="AdamW", lr0=0.001)

model_phase2 = YOLO(phase1_best_path)
model_phase2.train(data=data_yaml_path, epochs=85, patience=15, imgsz=640,
                   batch=24, optimizer="AdamW", lr0=0.0001)
```

4.4 Crack Model Implementation - Crack 3

The final crack detector was implemented independently from the pothole model. Crack 3 started from pretrained YOLO26m weights and used the processed Japan and United States crack-only datasets together with the synthetic low-light training set. Since thin cracks occupy a relatively small part of many road images, the model was trained at 800 x 800 input resolution

rather than 640 x 640. The larger input required a reduced batch size of 16 to remain within the available GPU memory.

Training was configured for a maximum of 100 epochs with AdamW, an initial learning rate of 0.0001, a three-epoch warm-up and early stopping with patience 15. No model modules were frozen during this run. The implementation also enabled horizontal flipping, HSV-based colour and brightness variation, mosaic augmentation and the framework training augmentations configured for the run. The best Crack 3 checkpoint was saved in the Crack3_Runs/crack3/weights directory and was used as the crack model in the final application. Performance values from this model are reported in Chapter 5 rather than in this implementation chapter.

4.5 Experimental Model Selection Interface

Before the two final models were fixed, a separate Flask-based testing interface was implemented to make development comparisons easier. A model registry stored the path, input size and class mapping for each trained checkpoint. The interface could therefore load one selected model at a time from a drop-down list without changing the remaining application code. This testing page included earlier YOLO11 models, Final 2 to Final 7, crack-only models and Crack 3, providing a practical way to inspect predictions from the different development stages on the same type of input.

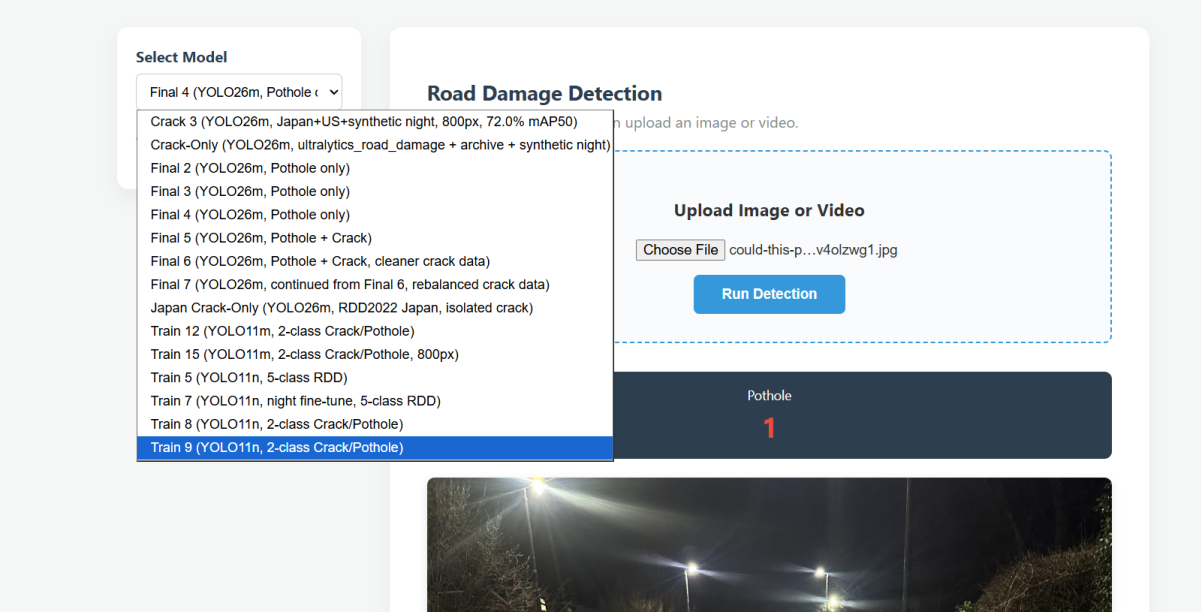


Figure 4.2: Experimental multi-model selection interface used during development

4.6 Final Dual-Model Web Application

After model selection was completed, the final application was simplified to use only the two specialised detectors: Final 4 for potholes and Crack 3 for cracks. The Flask application loads both weight files when it starts. A single uploaded image is then read once and passed independently to the two models. The pothole model receives a 640-pixel inference size and the crack model receives an 800-pixel inference size. Each model draws detections on its own copy of the original image, so predictions from one detector do not alter the input received by the other detector.

The web interface supports both image and video upload. For an image, the server returns two annotated result images together with separate pothole and crack counts. For video, each frame is processed separately by the two detectors and written to two result videos. A confidence threshold of 0.35 is applied in the application inference code. The final interface intentionally presents the pothole and crack outputs side by side, reflecting the decision to retain two specialised models rather than merge their detections into a single result image.

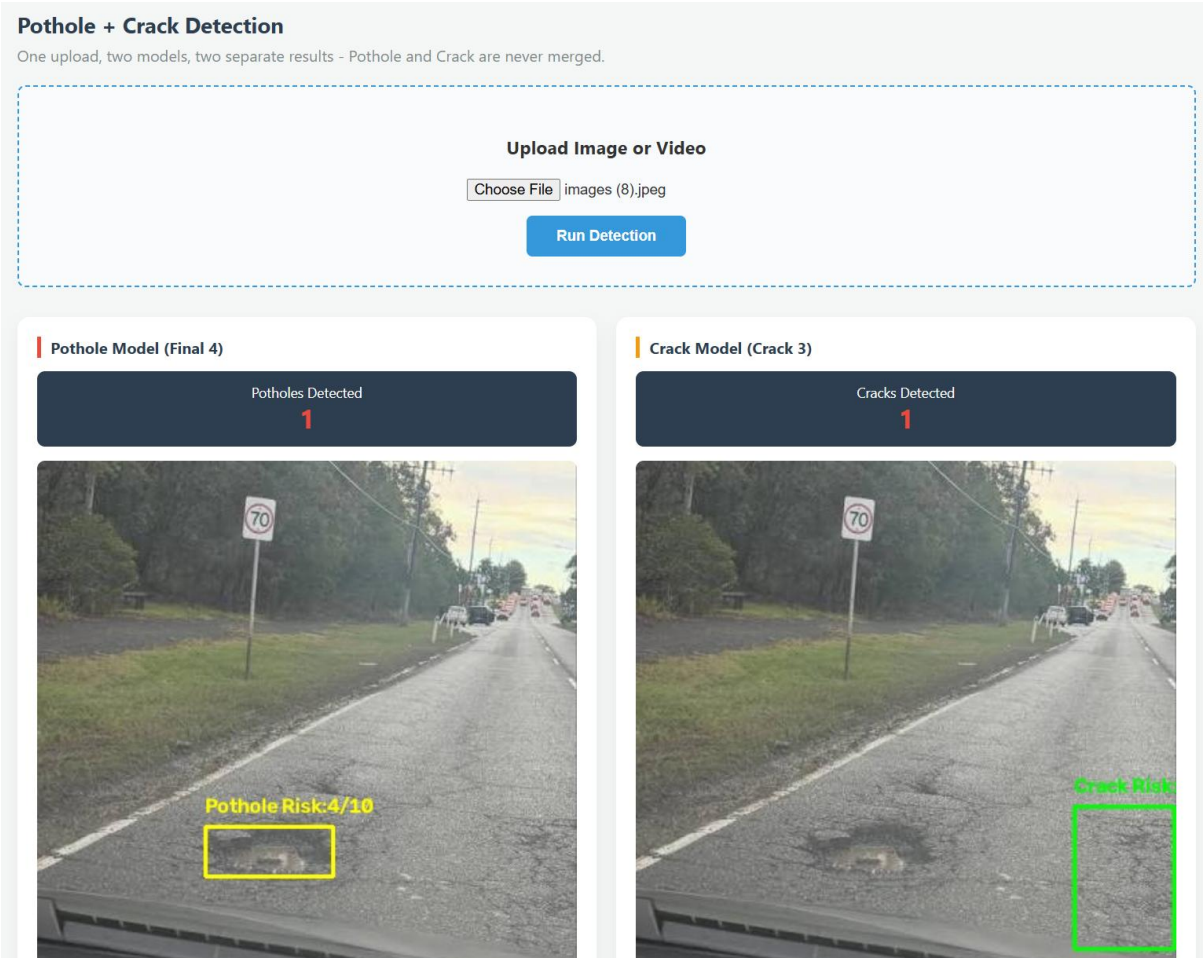


Figure 4.3: Final dual-model web application showing separate pothole and crack outputs

4.7 Automated Risk-Scoring Implementation

Risk scoring was integrated directly into the inference stage. For each predicted bounding box, the code first calculates the ratio between the bounding-box area and the complete image or video-frame area. A base weight of 4.0 is assigned to potholes and 2.0 to cracks. The area ratio is multiplied by 25 and capped at 6.0 to form a proximity/size contribution. The sum of the base weight and this contribution is multiplied by the model confidence, rounded and clipped to a value between 1 and 10.

Listing 4.4: Implemented rule-based risk score

```
area_ratio = box_area / frame_area
base_weight = 4.0 if is_pothole else 2.0
proximity_factor = min(area_ratio * 25.0, 6.0)
raw_score = (base_weight + proximity_factor) * confidence
risk = int(np.clip(np.round(raw_score), 1, 10))
```

The displayed colour is selected from the final score: values from 1 to 3 are shown in green, 4 to 6 in yellow and 7 to 10 in red. This processing takes place after each detection and before the annotated image or video frame is saved, allowing the user to see both the defect category and its associated risk score directly on the output.

4.8 Live Camera Detection

A separate OpenCV script was implemented for live testing. The script opens a single camera stream, including a phone camera connected to the computer, and loads Final 4 and Crack 3 at startup. Every captured frame is copied into two independent working images. One copy is processed by the pothole detector and the second by the crack detector using the same 0.35 confidence threshold and the same risk-scoring function as the web application.

Two OpenCV windows are then displayed simultaneously: one for pothole detection and one for crack detection. This arrangement was chosen so that the two models can be inspected independently during live operation and so that bounding boxes from one detector cannot obscure the output of the other. The live loop continues until the user presses the q key, after which the camera and both display windows are released.

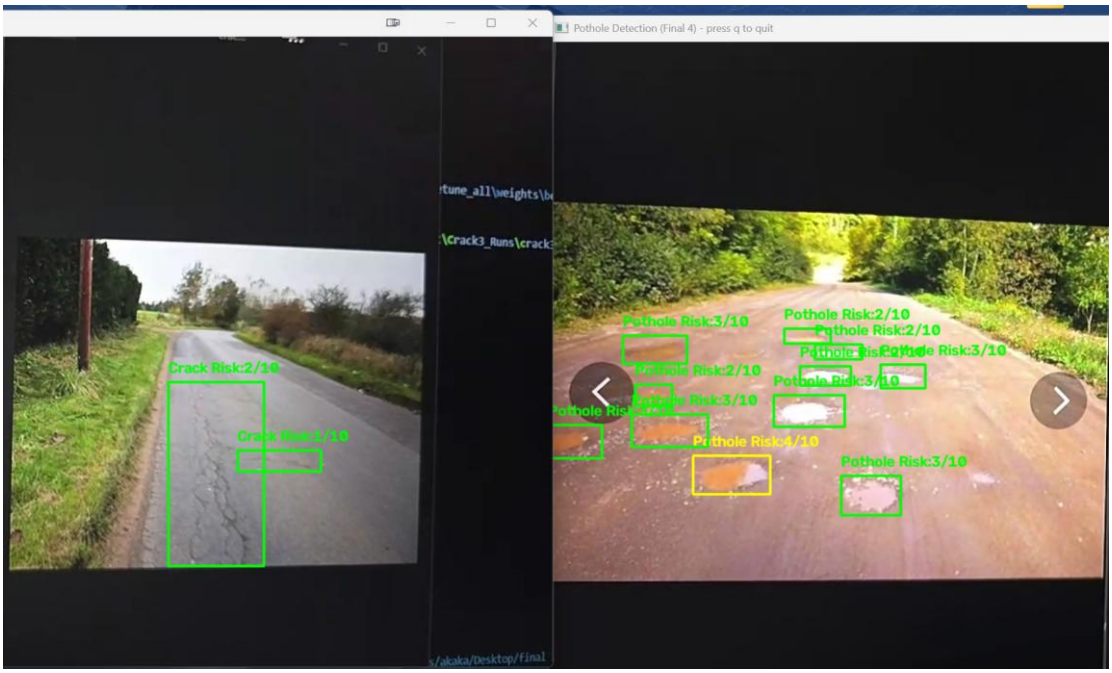


Figure 4.4: Live camera implementation showing separate crack and pothole detection windows

4.9 Model Compression Implementation

Post-training compression was applied after the final checkpoints had been selected. Final 4 and Crack 3 were exported from their best.pt checkpoints to ONNX using half-precision (FP16) weights and graph simplification. The export step therefore changed the deployment representation of the trained network without retraining it. Earlier dynamic and static INT8 experiments were also attempted on Final 4, but these remained experimental variants rather than the deployment choice.

Each FP16 ONNX model was loaded again through the Ultralytics/ONNX Runtime inference path and re-validated against the corresponding evaluation data. This allowed the original and compressed checkpoints to be compared using the same precision, recall, mAP50 and mAP50-

95 measures. The measured change in file size and detection performance, together with the limitations of the speed benchmark, are reported in Chapter 5.

4.10 Chapter Summary

This chapter translated the methodology into a working dual-model system comprising separate pothole and crack pipelines, YOLO26m detectors, a Flask interface, rule-based risk scoring, live camera inference and post-training compression. Chapter 5 evaluates the resulting models using quantitative metrics and qualitative detection examples.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 Chapter Overview

The analysis is based on the recorded Ultralytics validation outputs, the separate day and night pothole tests, the confidence and precision-recall curves, confusion matrices, qualitative prediction examples and the post-training compression experiments. The discussion also revisits the combined pothole-and-crack models because those experiments were important in deciding why the final application uses two specialist detectors rather than one shared model.

The two final models are Final 4 for pothole detection and Crack 3 for crack detection. They are evaluated independently because they were trained on different datasets and at different input resolutions the results are reported using precision, recall, mAP50 and mAP50-95, followed by a qualitative review of the kinds of detections that were successful and the cases that remained difficult.

5.2 Evaluation Setup and Scope

The results reported in this chapter are based on the best saved model, best.pt. During training, the model was checked on the validation set after end of each epoch, and the best-performing version was saved. The precision, recall and mAP values therefore show how well the final saved model performed on validation images, while the training loss only shows how well it learned the training data

5.1: Final model evaluation configuration

Model	Task	Input size	Validation images processed	Validation instances	GPU inference time
Final 4	Pothole detection	640 × 640	5,631	11,351	7.1 ms/image
Crack 3	Crack detection	800 × 800	1,530	2,556	8.9 ms/image

For Final 4, 5 of the 5,636 validation images were skipped because their labels mixed segmentation and detection formats, so the reported results are based on 5,631 processed images. Crack 3 used all 1,530 validation images, with one duplicate label removed automatically. These details are reported to make the evaluation transparent. The inference times in Table 5.1 were measured using the Ultralytics validation routine on the NVIDIA RTX 5080 Laptop GPU.

5.3 Model Development and Selection

The final system was developed through several experiments rather than one training run. The early Train 5–Train 15 models were used to test the data pipeline, YOLO11 variants, and different class setups. The later Final-series and dedicated crack models were then compared to select the two models used in the final system.

Table 5.2: Experimental model development summary

Model	Architecture / task	Input size	mAP50	Dataset(s) used	Total images	Role / observation
Train 5	YOLO11n, 5 classes	640	0.715	Japan daytime road-damage dataset (5-class RDD)	13,133	Early daytime multi-class baseline.
Train 7	YOLO11n, 5 classes	640	0.877	night_dataset (continued from Train 5)	3,685	Night fine-tuning experiment; useful development result.
Train 8*	YOLO11n, 2 classes	640	0.736	japan_daytime + night_time + pothole_only	~19,589*	Baseline: YOLO11n, Auto optimizer, batch 16.
Train 9*	YOLO11n, 2 classes	640	0.614	japan_daytime + night_time + pothole_only	~19,589*	Changed to **AdamW** and batch **32**.
Train 12*	YOLO11m, 2 classes	640	0.971	japan_daytime + night_time + pothole_only	~19,589*	Changed to **YOLO11m**, Auto optimizer, 150 epochs.
Train 15*	YOLO11m, 2 classes	800	0.809	japan_daytime + night_time + pothole_only	~19,589*	Changed to 800px, AdamW, 100 epochs, freeze 10 layers.
Final 2	YOLO26m, pothole only	640	0.824	New pothole detection.v2i.yolo26 only	15,696	Clean pothole result on a narrower dataset; intended multi-source combination did not load as planned.
Final 3	YOLO26m, pothole only	640	0.780	HRP4K + New pothole detection.v2i.yolo26 + night_time + Pothole.v1i.yolo26	21,699	Four pothole data sources combined through manifests.
Final 4	**YOLO26m, pothole only**	**640**	**0.764**	**gravel + HRP4K + New pothole detection.v2i.yolo26 + night_time + Pothole.v1i.yolo26**	**28,773**	**Five-source pothole model selected for the final system.**
Final 5	YOLO26m, pothole + crack	640	0.560	Final 4 pothole sources + road_crack_clean	36,731	Pothole remained strong; crack performance was weak.
Final 6	YOLO26m, pothole + crack	640	0.622	Final 4 pothole sources + ultralytics_road_damage	32,094	Best combined run, but crack remained the limiting class.
Final 7	YOLO26m, pothole + crack	640	0.567	HRP4K + night_time + Pothole.v1i.yolo26 + archive_split + smartathon_clean	23,708	Rebalancing did not resolve the crack limitation.
Crack-only	YOLO26m, crack only	640	0.441	ultralytics_road_damage + archive_split + synthetic night copies	9,329	First dedicated crack model.
Crack 2	YOLO26m, Japan crack only	640	0.615	Japan crack-only dataset (10,506 train-folder images + 2,627 test images)	13,133	Japan-only crack model; clear improvement over Crack-only.

Model	Architecture / task	Input size	mAP50	Dataset(s) used	Total images	Role / observation
Crack 3	YOLO26m, crack only	800	0.720	Japan + United States crack-only data + synthetic night copies	32,919	Final crack specialist used in the deployed system.

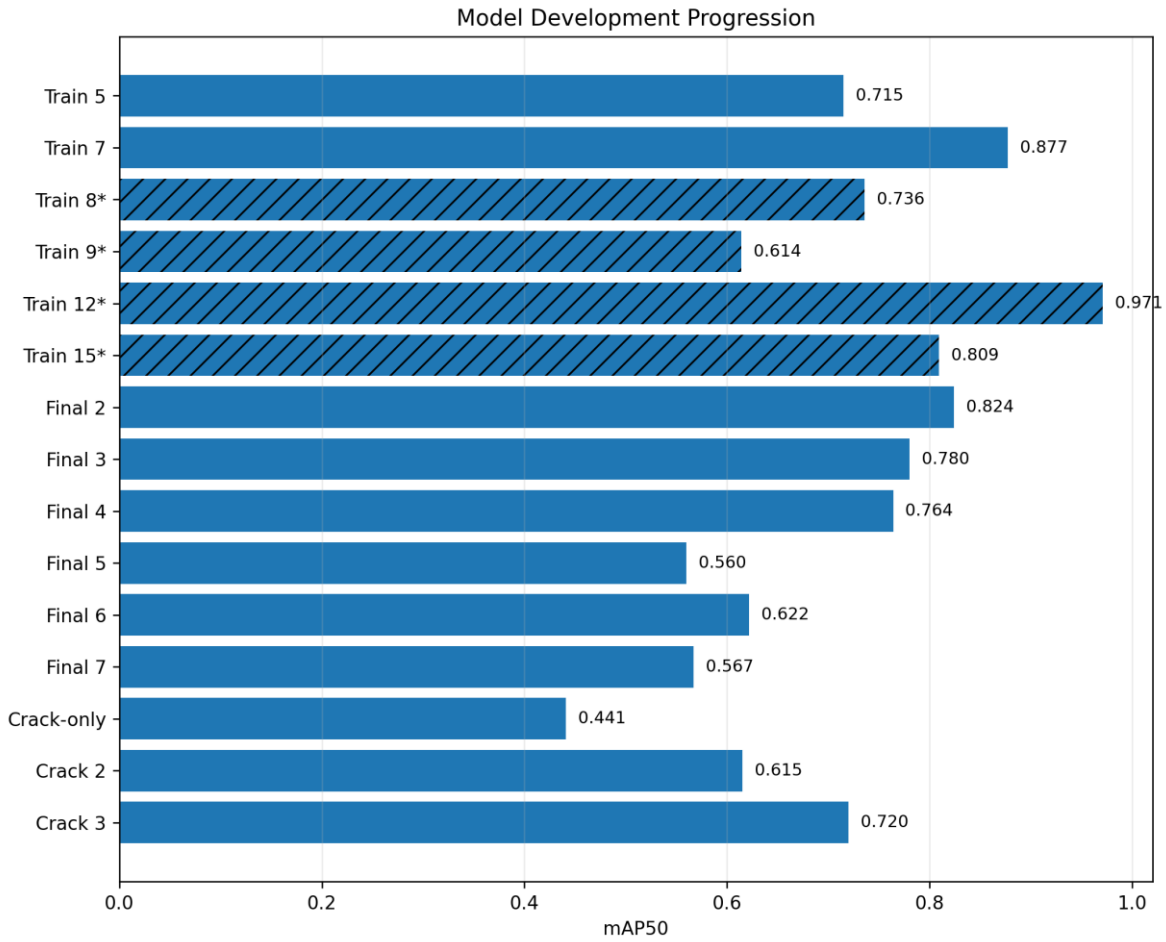


Figure 5.1: mAP50 progression across the main model-development experiments

5.3.1 Early YOLO11 Development

Figure 5.1 places the early YOLO11 experiments alongside the later YOLO26m runs. Train 5 and Train 7 were useful baseline experiments, while Train 8, Train 9, Train 12 and Train 15 were later found to contain validation leakage. This is why the very high 0.971 mAP50 from Train 12 is not treated as the best model in this thesis. Keeping these runs in the results chapter is still important because they show how the project developed and, more importantly, why the evaluation procedure was tightened before selecting the final models.

5.3.2 Pothole and Combined-Model Progression

The clean pothole sequence moved from 0.824 mAP50 in Final 2 to 0.780 in Final 3 and 0.764 in Final 4 as the training sources became more diverse. The lower score was accepted because Final 4 represented the broadest intended operating domain, including day, night and gravel or unpaved imagery. The combined Final 5 to Final 7 experiments then showed a different problem: pothole detection remained comparatively strong, but crack detection remained much

weaker. Final 6 was the best combined run overall, yet its crack mAP50 was only 0.465. This repeated class imbalance was the main reason the final system moved to two specialist detectors.

5.3.3 Dedicated Crack Model Progression

The crack pipeline improved in three clear stages. The first dedicated Crack-only model achieved 0.441 mAP50. Crack 2 then isolated the Japan crack data and improved mAP50 to 0.615, with precision of 0.643, recall of 0.557 and mAP50-95 of 0.302 on 1,050 validation images containing 1,458 crack instances. Crack 3 extended the crack training data to Japan and United States imagery, added synthetic low-light copies for training, increased the input size to 800 pixels and reached 0.720 mAP50 in the explicit validation summary. The progression is shown in Figure 5.2.

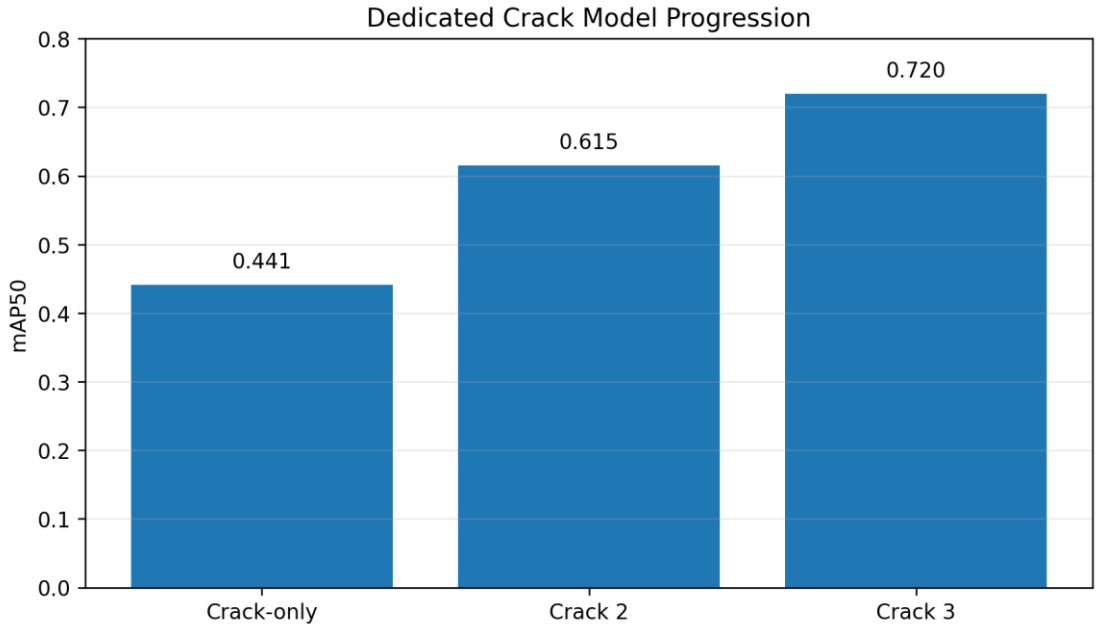


Figure 5.2: Dedicated crack-model progression from Crack-only to Crack 3

Crack 2 was an important intermediate result because it showed that a cleaner, single-purpose crack dataset could substantially improve performance before the final Crack 3 configuration was introduced. Figure 5.3 shows a representative Crack 2 validation batch. The predictions capture several visible cracked regions, but the examples also show missed or fragmented defects and occasional boxes around road textures or markings. Those observations motivated the larger and more varied Crack 3 training pipeline.

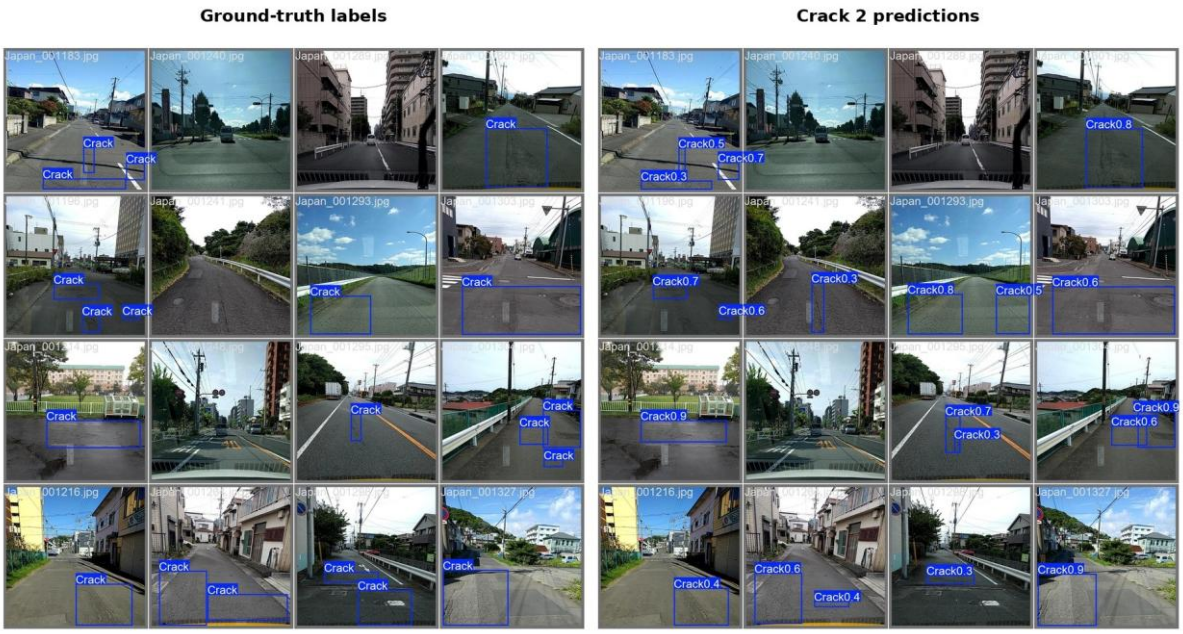


Figure 5.3: Crack 2 qualitative validation example - ground-truth labels and model predictions

5.4 Final Pothole Model Results - Final 4

5.4.1 Overall Validation Performance

Table 5.3: Final 4 validation performance

Metric	Result
Precision	0.826
Recall	0.698
mAP50	0.764
mAP50-95	0.462
Peak F1	0.76

Final 4 produced a validation mAP50 of 0.764 with precision of 0.826 and recall of 0.698. The difference between precision and recall is useful in understanding the model. It was more conservative about making a positive prediction than it was exhaustive about finding every pothole. In practical terms, when it drew a pothole box it was usually correct, but some smaller, distant or visually ambiguous defects were still missed.

5.4.2 Training Behaviour

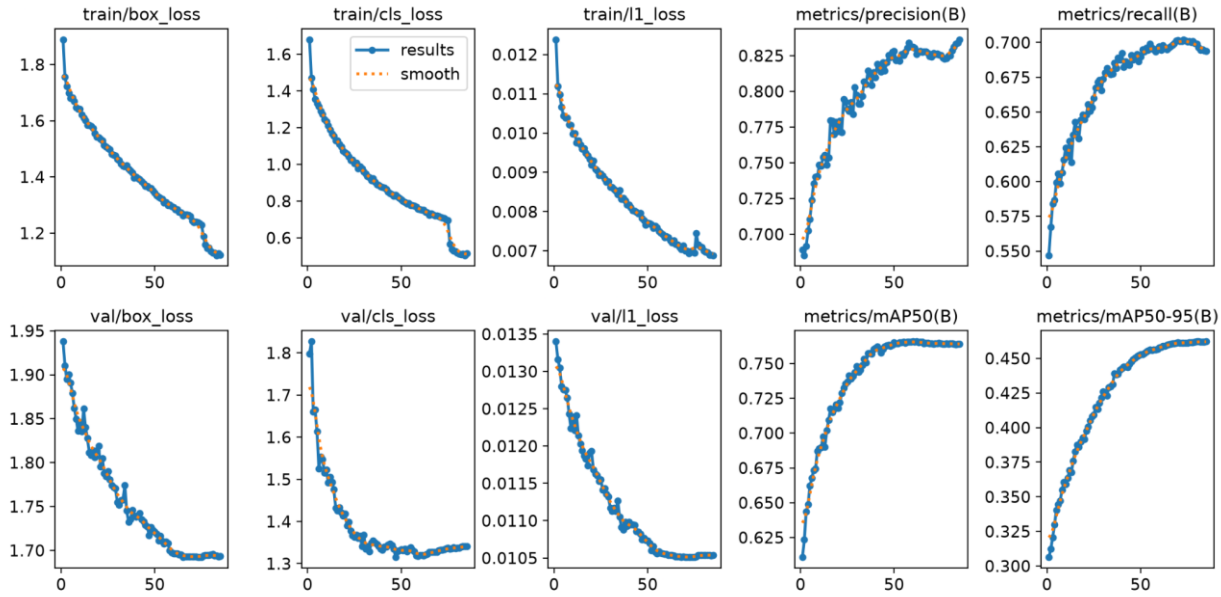


Figure 5.4: Final 4 Phase 2 training and validation curves

Figure 5.4 shows the 85-epoch full-network fine-tuning phase that followed the frozen-backbone warm-up. Training box, classification and localization losses continued to decrease, while the validation losses reduced and then largely flattened. At the same time, precision rose to approximately 0.83, recall settled close to 0.70, and both mAP curves approached a plateau. The evaluation metrics became stable. Later epochs produced only small changes, suggesting that the model had reached the practical limit of this data and configuration rather than continuing to gain large improvements from additional training.

5.4.3 Confidence and Precision-Recall Behaviour

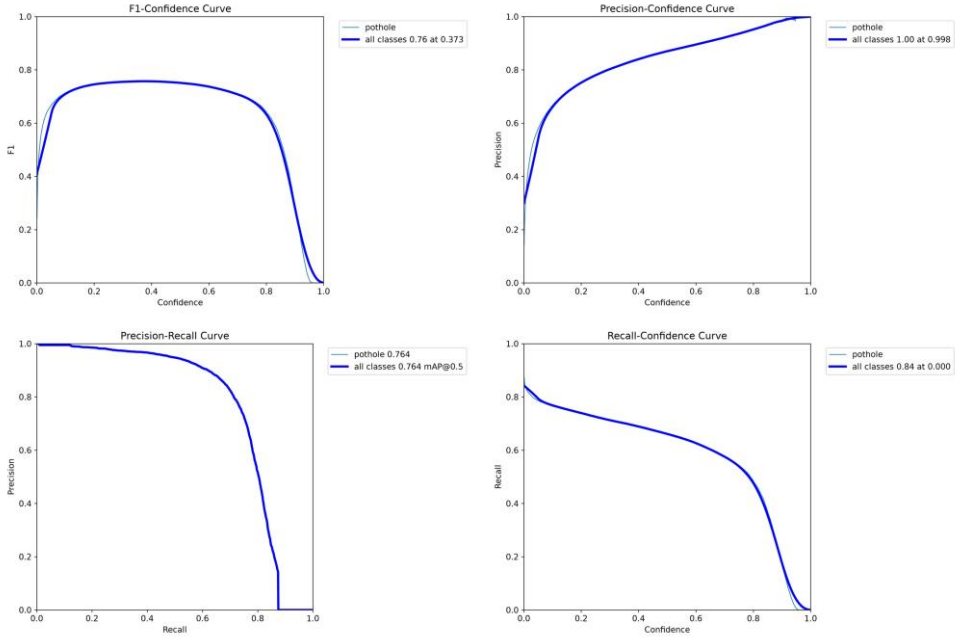


Figure 5.5: Final 4 F1-confidence, precision-confidence, precision-recall and recall-confidence curves

The confidence curves provide a more useful deployment view than a single score. The F1 curve peaks at approximately 0.76 when confidence is 0.373. Precision rises steadily as the threshold increases, while recall falls because more low-confidence detections are discarded. The precision-recall curve gives an area corresponding to mAP50 = 0.764. Together, these curves show why a confidence threshold around the mid-0.30 range is sensible for the final application: moving much higher would improve precision but would begin to remove a noticeable number of true potholes.

5.4.4 Confusion Matrix Analysis

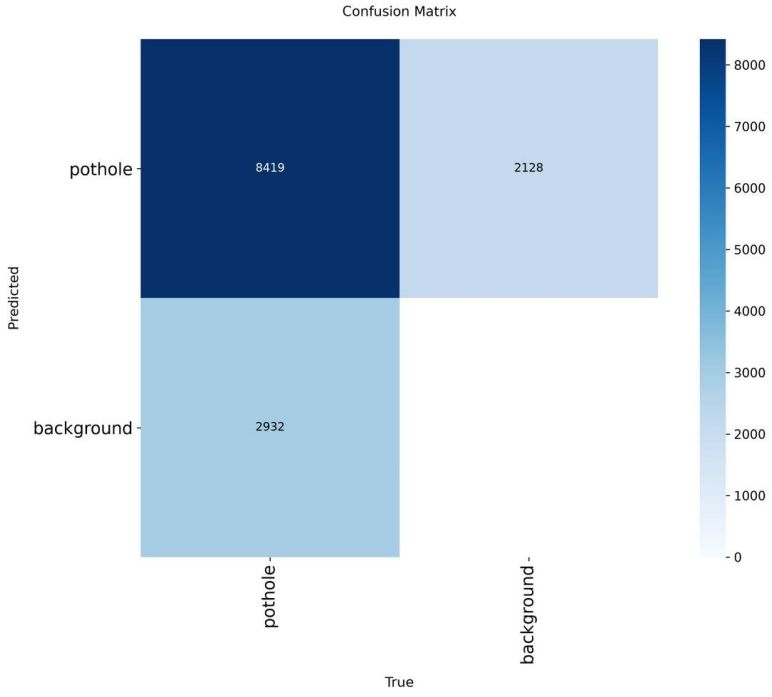


Figure 5.6: Final 4 raw confusion matrix

The raw confusion matrix records 8,419 matched pothole detections, 2,932 pothole instances that were missed, and 2,128 unmatched pothole predictions. In an Ultralytics object-detection confusion matrix, the background row and column are bookkeeping for unmatched ground-truth objects and unmatched predictions; background is not a trained pothole class. For that reason, the raw-count matrix is reported here because it is easier to interpret for this single-class detector than the normalized version.

. Large, obvious potholes are often detected with strong confidence, while the missed cases are more likely to be small, distant, poorly illuminated or visually mixed with patches and road texture. These difficult cases have a larger effect on recall than they do on the appearance of a few hand-picked examples.

5.4.5 Day and Night Test Performance

Table 5.4: Final 4 day and night test results

Subset	Images processed	Instances	Precision	Recall	mAP50	mAP50-95
Day	2,890	5,214	0.794	0.670	0.728	0.439
Night	372	671	0.878	0.861	0.895	0.530

Table 5.5: Final 4 performance by dataset source

Dataset source	Images processed	Instances	Precision	Recall	mAP50	mAP50-95
Gravel	659	1,000	0.753	0.679	0.692	0.346
HRP4K	900	921	0.609	0.423	0.446	0.235
Smartathon / New pothole	1,054	2,645	0.848	0.749	0.812	0.531
Night-time	372	671	0.878	0.861	0.895	0.530
Pothole.v1 / day_v1	277	648	0.785	0.751	0.805	0.484

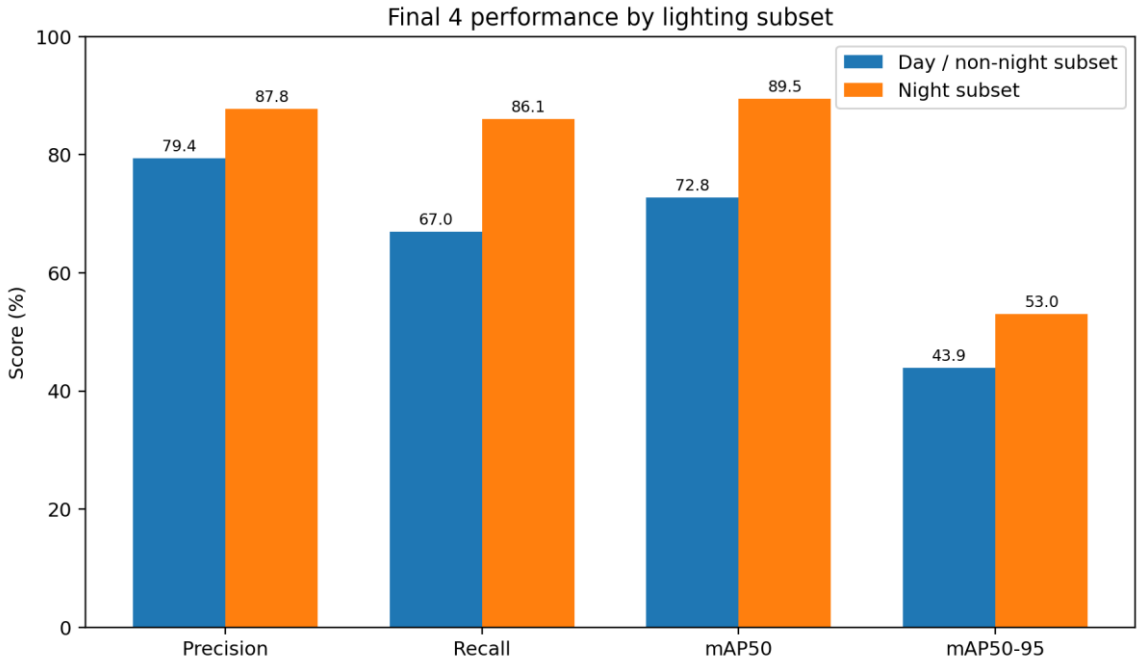


Figure 5.7: Final 4 performance on day/non-night and night test subsets

The night-only result was one of the strongest outcomes of the pothole experiments. Final 4 achieved 0.895 mAP50 on the dedicated night subset, compared with 0.728 on the group treated as day/non-night. This does not mean that potholes are intrinsically easier to detect at night. The night data came from a dedicated, comparatively consistent source, while the non-night

group combined several visually different datasets, including the more difficult HRP4K and gravel material. The difference therefore reflects dataset composition as well as lighting.

The day/night split is also approximate. The notebook classified the dedicated night_time dataset as night and treated every other source as day. If a gravel source contained an occasional dark image, it remained in the non-night group because the folder structure did not provide a reliable lighting label

5.4.6 Annotation Distribution and Qualitative Pothole Results

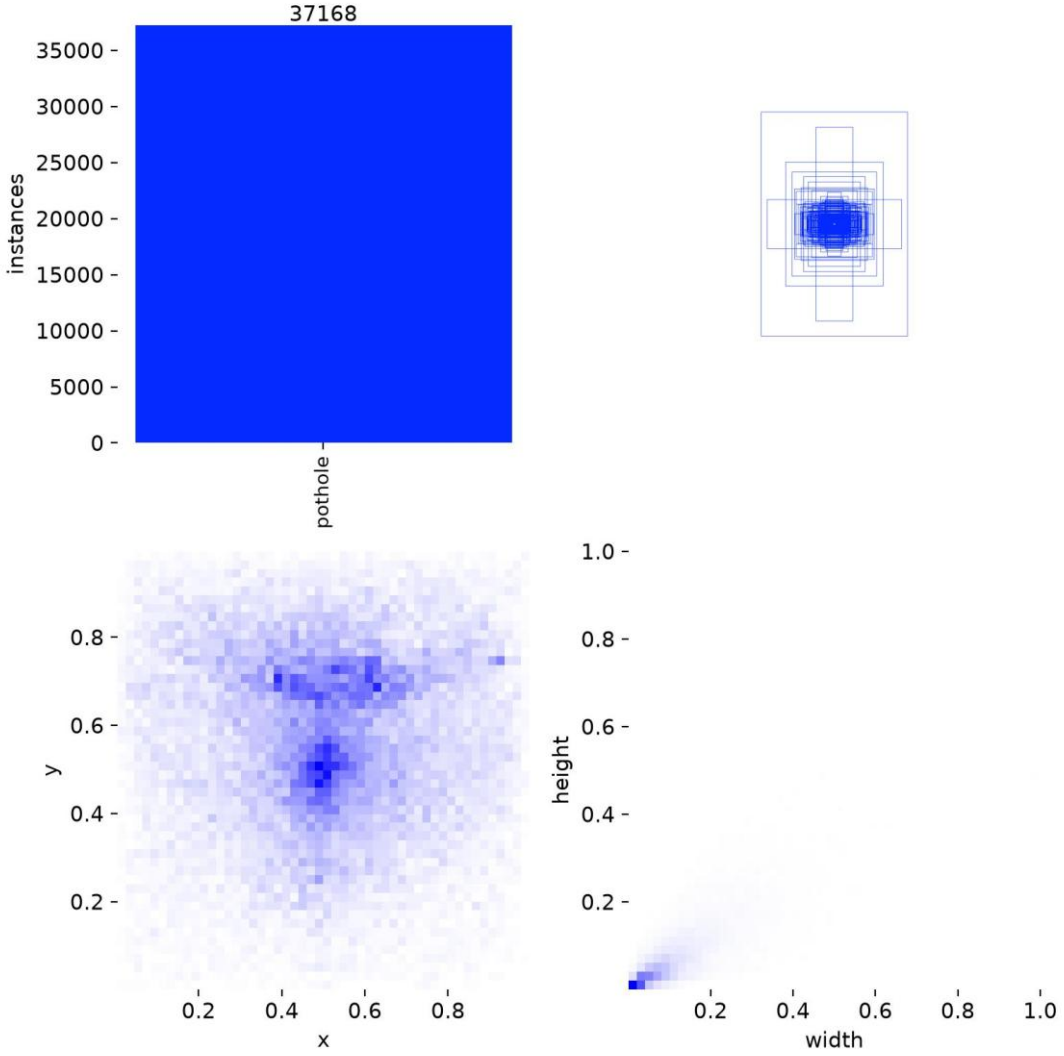


Figure 5.8: Final 4 pothole annotation distribution and bounding-box geometry

The annotation plot shows that potholes are usually small relative to the complete road image. Most boxes occupy a limited fraction of the frame, with many positioned in the central and lower road region. This matters because small objects provide fewer pixels for the detector to use, particularly when the damage is far from the camera. The distribution therefore helps explain why mAP50 is noticeably higher than mAP50-95: detecting that a pothole exists can be easier than placing a very precise bounding box around a small irregular boundary.



Figure 5.9: Final 4 qualitative validation example - ground-truth annotations followed by model predictions

The qualitative validation examples show that Final 4 can detect potholes across markedly different scenes, including dark frames, ordinary paved roads, rough surfaces and images captured from different viewpoints. The predictions are strongest when the damage has a clear boundary and occupies a reasonable number of pixels. where road patches, shadows or rough texture resemble pothole edges. These examples are consistent with the quantitative result the model is useful and reasonably precise, but it is not a complete substitute for manual inspection in every scene.

5.5 Final Crack Model Results - Crack 3

5.5.1 Overall Validation Performance

Table 5.6: Crack 3 validation performance

Metric	Result
Precision	0.693
Recall	0.654
mAP50	0.720
mAP50-95	0.394
Peak F1	0.67

Crack 3 achieved 0.720 mAP50, with precision of 0.693 and recall of 0.654 on 1,530 validation images containing 2,556 crack instances. The precision-recall plot labels the integrated mAP50 as 0.721 because of rounding in the plotting output; the explicit validation summary printed 0.720. Both refer to the same model and evaluation run.

The dedicated crack results show a steady improvement rather than a single jump. The first Crack-only model achieved 0.441 mAP50, Crack 2 increased this to 0.615, and Crack 3 reached 0.720 in the explicit validation summary. The improvement cannot be attributed to input size alone. Across these iterations the crack data were cleaned and reorganised, Crack 2 isolated the Japan crack subset, and Crack 3 expanded the source coverage to Japan and United States imagery, added synthetic low-light training copies and increased the input size to 800 pixels.

5.5.2 Training Behaviour

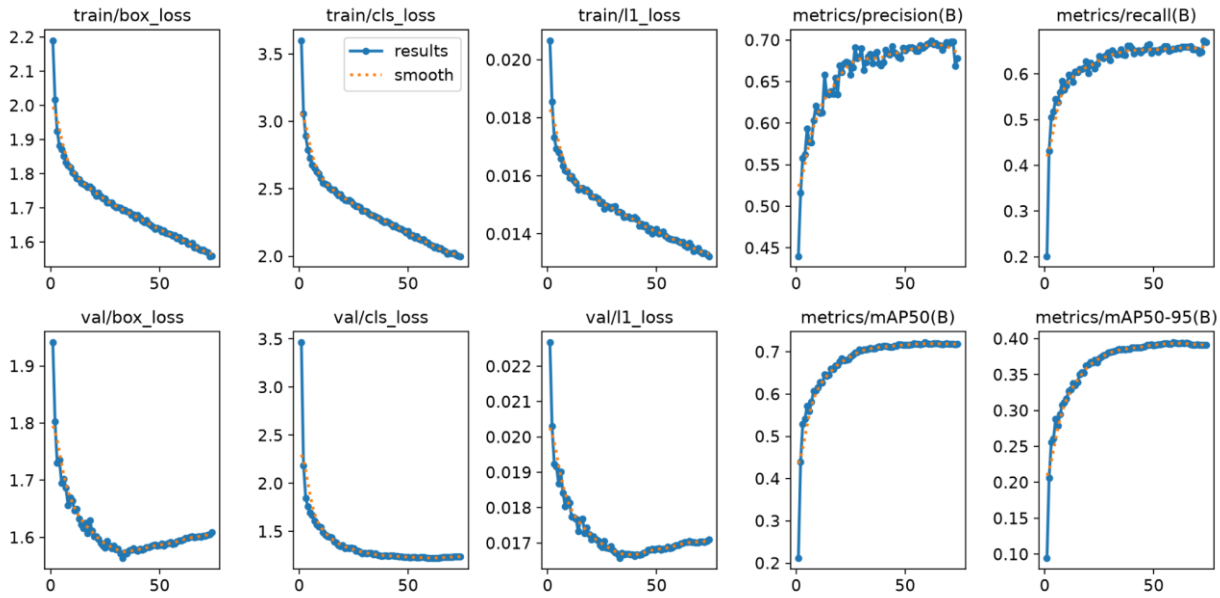


Figure 5.10: Crack 3 training and validation curves

Crack 3 was configured for a maximum of 100 epochs but stopped after 74 because no further improvement had been observed for 15 epochs. The best checkpoint was recorded at epoch 59. Figure 5.10 makes the reason visible: the training losses continued to fall, while the validation box and localization losses reached their lowest region earlier and then began to rise slightly.

At the same time, mAP50 and mAP50-95 had already flattened. This is a mild sign of overfitting after the useful learning phase, and the early-stopping mechanism prevented the final model from simply taking the last epoch.

5.5.3 Confidence and Precision-Recall Behaviour

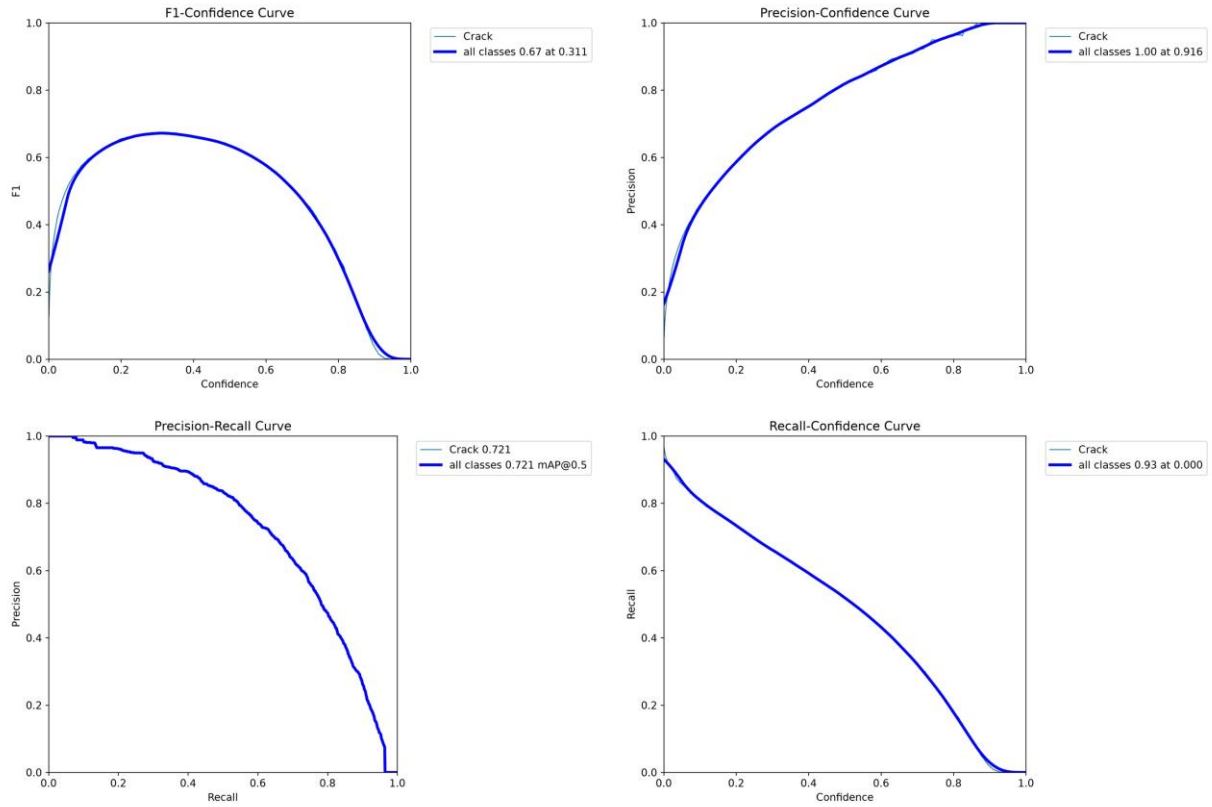


Figure 5.11: Crack 3 F1-confidence, precision-confidence, precision-recall and recall-confidence curves

Crack detection benefits from a slightly lower confidence threshold than pothole detection. The F1 curve reaches its maximum of about 0.67. Precision then increases with confidence but recall declines quickly because faint or fragmented cracks are often assigned moderate rather than very high confidence. This behaviour is expected for thin defects whose visual appearance can be close to repaired seams, lane markings, shadows or ordinary asphalt texture.

5.5.4 Confusion Matrix Analysis

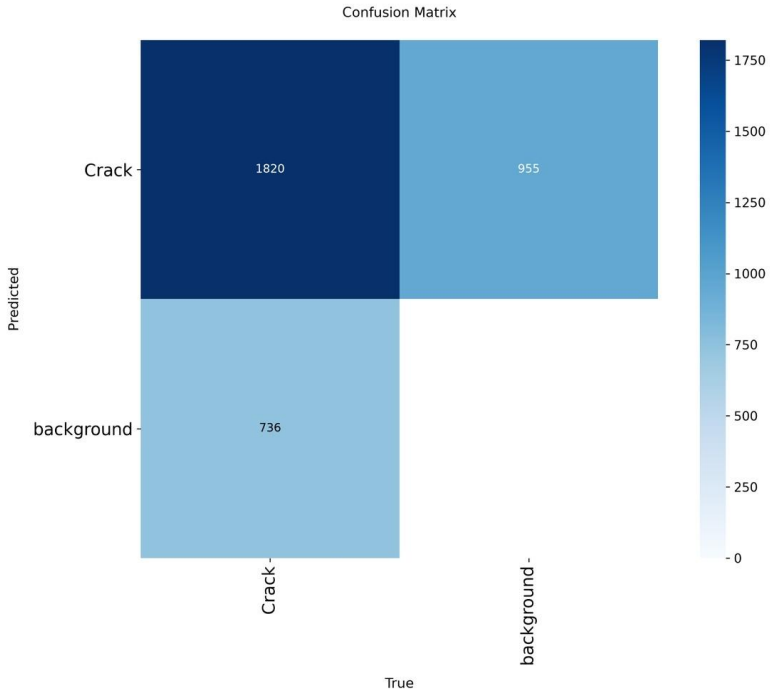


Figure 5.12: Crack 3 raw confusion matrix

The Crack 3 confusion matrix records 1,820 matched crack detections, 736 missed crack instances and 955 unmatched crack predictions. As with Final 4, the background row and column represent unmatched objects and predictions rather than a separate class learned by the model. Only the raw-count matrix is therefore retained in the main results because it communicates the error counts more clearly for a single-class crack detector.

The balance of errors is informative. Crack 3 still produces both missed defects and false crack proposals, which is consistent with its lower precision and recall than Final 4. Unlike a pothole, a single crack may be long, thin, branched or broken into several visible segments. A rectangular bounding box can therefore contain a large amount of undamaged road around the crack, making exact localization more difficult and contributing to the lower mAP50-95 score.

5.5.5 Annotation Distribution and Qualitative Crack Results

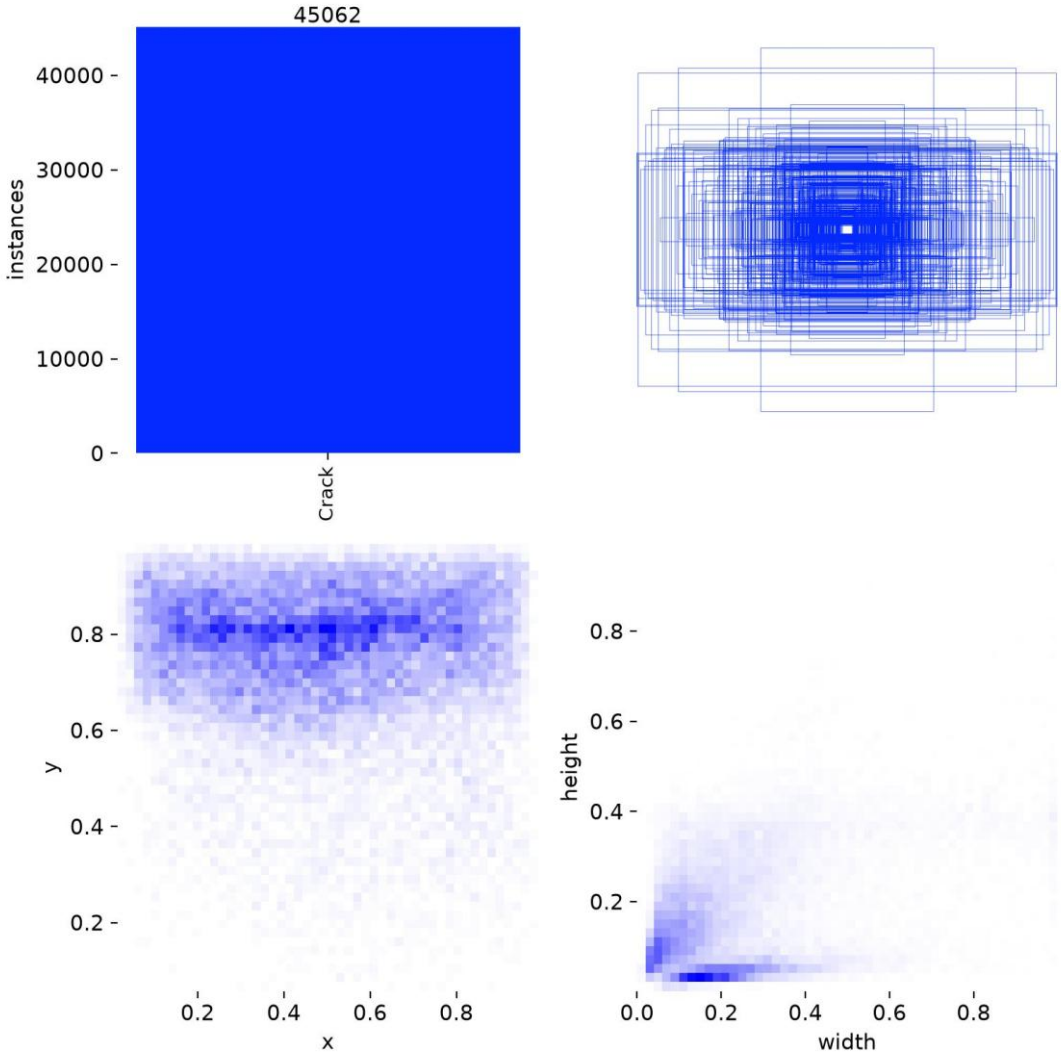


Figure 5.13: Crack 3 annotation distribution and bounding-box geometry

The Crack 3 label distribution is visibly more varied than the pothole distribution. The training plot contains 45,062 crack instances and includes narrow vertical boxes, long horizontal boxes and larger regions covering groups of connected cracks. This range of shapes is one reason crack detection is difficult: the detector is not learning one consistent object silhouette, but a family of elongated and irregular damage patterns.

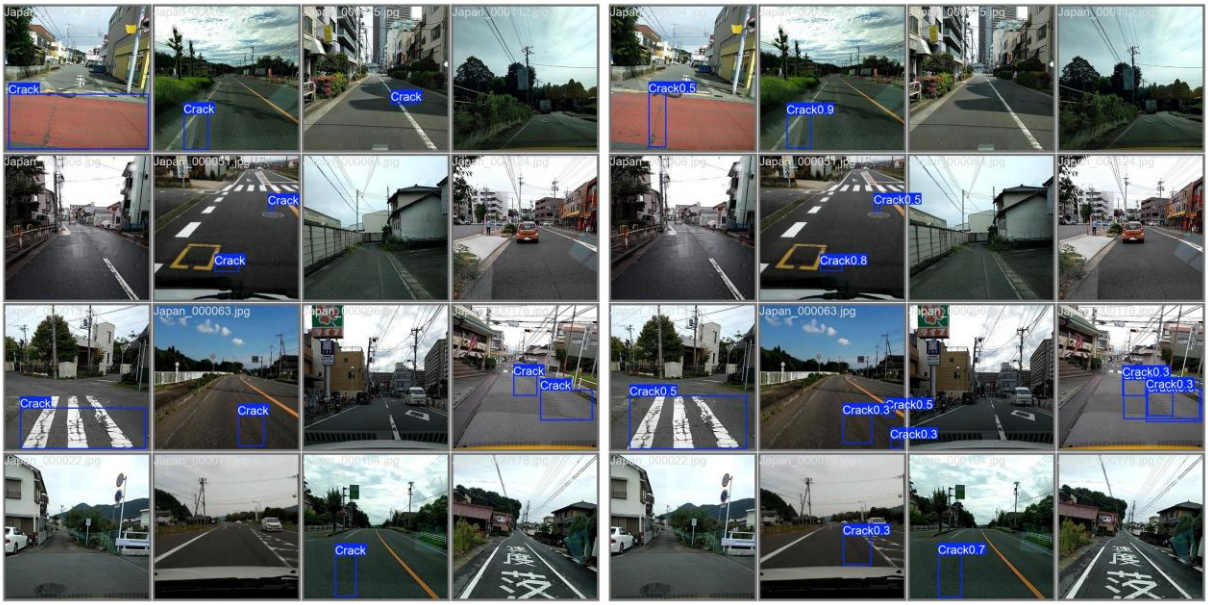


Figure 5.14: Crack 3 qualitative validation example - ground-truth annotations and model predictions

The qualitative examples show useful detections on longitudinal, branching and broader cracked regions, but also illustrate the remaining failure modes. Some faint cracks are missed, while road markings, repaired surfaces and high-contrast texture can occasionally attract a crack box. The model also sometimes divides one damaged region into several detections or uses a large box around an irregular crack group. These behaviours are visible in the validation images and agree with the gap between mAP50 and the stricter mAP50-95 measure.

A further limitation is that the Crack 3 validation and test splits contain the original Japan and United States imagery; the synthetic low-light images were used only for training. This is a deliberate choice because evaluating on synthetic copies of training-style transformations would overstate nighttime performance The low-light augmentation is evidence of an attempt to improve robustness, not a replacement for a genuinely labelled nighttime crack benchmark.

5.6 Combined Models Compared with Specialist Models

Table 5.7: Per-class mAP50 in the combined pothole-and-crack models

Model	Overall mAP50	Pothole mAP50	Crack mAP50
Final 5	0.560	0.774	0.345
Final 6	0.622	0.779	0.465
Final 7	0.567	0.801	0.333

Table 5.7 shows that the problem with the combined architecture was not pothole detection. Across all three combined runs, pothole mAP50 stayed between 0.774 and 0.801. The difficult class was consistently crack, which ranged from 0.333 to 0.465. The dedicated crack sequence then improved from 0.441 in Crack-only to 0.615 in Crack 2 and 0.720 in Crack 3. Final 6 was the best combined compromise, but the staged specialist results show why the final system retained separate pothole and crack detectors.

The comparison is not a perfectly controlled ablation because Crack 3 used a different crack dataset and an 800-pixel input size. It would therefore be inaccurate to claim that model separation alone produced the entire improvement. What the experiments do show is that repeated attempts to make a shared 640-pixel detector handle both classes did not solve the crack weakness. Moving to a dedicated crack model created the freedom to change the data, resolution and training recipe without disturbing a pothole detector that was already working well. That practical result is the main justification for the final two-model architecture.

5.7 Model Compression Results

Table 5.8: Original and FP16 ONNX model comparison

Model	Representation	Size (MiB)	Precision	Recall	mAP50	mAP50-95
Final 4	PyTorch best.pt	42.0	0.826	0.698	0.764	0.462
Final 4	FP16 ONNX	39.0	0.831	0.683	0.755	0.450
Crack 3	PyTorch best.pt	42.01	0.693	0.654	0.720	0.394
Crack 3	FP16 ONNX	39.08	0.687	0.633	0.697	0.366

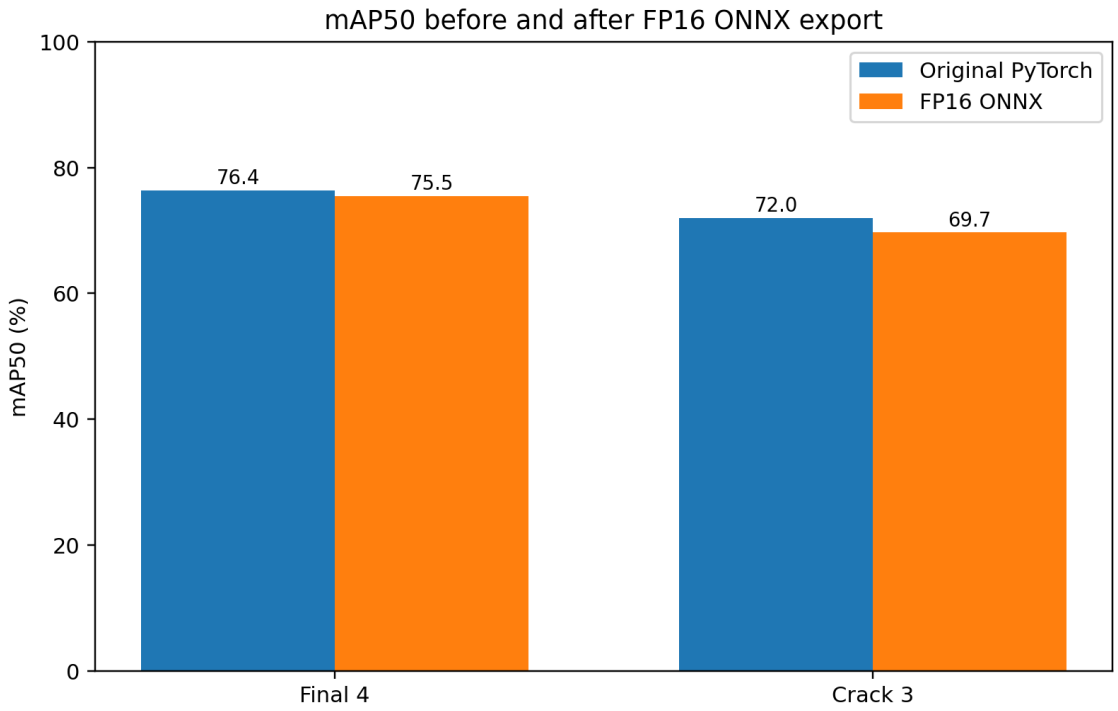


Figure 5.15: mAP50 before and after FP16 ONNX export

FP16 export preserved most of the pothole model performance. Final 4 mAP50 decreased by 0.009, from 0.764 to 0.755, while mAP50-95 decreased by 0.012. Precision changed slightly upward and recall fell by 0.015. For Crack 3 the effect was larger: mAP50 fell by 0.024 and mAP50-95 by 0.028, with recall decreasing by 0.021. The crack detector therefore appears more sensitive to reduced numerical precision than the pothole detector.

The file-size reduction was modest in both cases: about 7.0%, from approximately 42.0 MiB to 39.0 MiB. FP16 is often described as halving numerical precision, but that does not mean the complete serialized model file will necessarily become half the size. In this project, the actual storage saving was much smaller than the theoretical change in parameter precision.

INT8 quantisation was also attempted, but the evaluation crashed in the current setup and therefore no reliable performance value could be obtained. The FP16 ONNX model completed successfully, but its measured speed was only 3.1 FPS on CPU, which was too slow for practical live detection. Although the original PyTorch model reached about 100.7 FPS on GPU, these values are not directly comparable because they were measured using different execution backends and hardware. Therefore, no claim of speed improvement from compression is made.

5.8 Real-World System Evaluation

5.8.1 Deployment Confidence and Automated Risk Score

The final web application uses a confidence threshold of 0.35. The final pothole model achieved an mAP50 of 0.764, while the final crack model achieved an mAP50 of 0.720, showing reliable overall detection performance. A shared threshold of 0.35 was selected as a practical operating point for both models, allowing lower confidence but potentially valid road damage to remain visible without producing excessive detections.

After a defect is detected, the application assigns a risk score from 1 to 10 using the detected class, the apparent bounding-box area relative to the frame, and the model confidence. Potholes receive a higher base weight than cracks, while larger detected areas increase the risk score. The score is displayed using low, medium, and high colour bands. However, it should be interpreted as a rule-based hazard indicator rather than a physically measured severity value, since the dataset does not contain ground-truth information such as pothole depth, crack width, or engineering condition ratings.

5.8.2 Image, Video and Live-Camera Operation

The completed system was tested with uploaded images, uploaded video and a live camera stream. In each mode the same input is passed independently to Final 4 and Crack 3, and the two annotated outputs remain separate. This made it possible to inspect each specialist detector without overlapping boxes from two networks on one image. The live script also opened separate pothole and crack windows from the same camera source, including a phone camera used as the input device.

The original uncompressed PyTorch models showed strong real-time performance on the laptop GPU. Final 4 recorded a validation latency of 7.1 ms per image, Crack 3 recorded 8.9 ms, and the original GPU speed test reached approximately 100 FPS. The dual-model live camera system also operated smoothly during practical testing, demonstrating that the uncompressed models were suitable for live detection. In contrast, the FP16 ONNX compressed model achieved only about 3.1 FPS when executed through the available CPU backend, which was too slow for practical live detection. GPU testing of the compressed ONNX model could not be completed because of a CUDA device-binding error. Therefore, although FP16 compression retained most of the detection performance, inference speed was a disadvantage in the available deployment setup.

5.9 Comparison with Existing Studies

Table 5.9: Contextual comparison with selected published road-damage studies

Study / model	Task and data	Precision	Recall	mAP50	mAP50-95 / stricter metric
Final 4 (this project)	**Pothole detection across mixed daytime, nighttime and varied road conditions**	**0.826**	**0.698**	**0.764**	**0.462**
Tsanaullailla et al. (2024)	Nighttime pothole detection, 3,685 images	0.830	0.879	0.899	0.546
Crack 3 (this project)	**Crack detection using Japan and US road images**	**0.693**	**0.654**	**0.720**	**0.394**
Verma et al. (2025)	Dedicated YOLO11x-GhostConv crack detector	0.630	0.440	0.490	0.210
Wan et al. (2022), YOLO-LRDD	Multi-class road-damage detection on RDDC	0.592	0.582	0.576	0.306 reported as mAP0.95

Final 4 achieved an overall mAP50 of 0.764 across a much broader and more diverse pothole dataset than the nighttime-only study by (Tsanaullailla *et al.*, 2024) Unlike studies trained on a single road condition or one dedicated dataset, Final 4 was developed using multiple sources covering daytime, nighttime, gravel/unpaved roads and different road environments, with nearly 28,773 image references in the final dataset. This makes the task more challenging but also gives the model broader exposure to varied conditions and supports better robustness across different environments.

Crack 3 also achieved a higher mAP50 of 0.720 than the 0.490 reported by (Verma *et al.*, 2025). The comparison is particularly relevant because both approaches use specialist models rather than relying on one combined detector. However, Crack 3 was trained using a considerably larger and more varied crack dataset from Japan and the United States, including additional low-light data, which likely contributed to its stronger and more robust performance. (Verma *et al.*, 2025). reported lower overall crack performance and also noteand single combined model was unsatisfactory.

Overall, the main strength of the proposed system is not only its mAP50, but the fact that both specialist models were trained on larger and more diverse datasets than several of the reference studies. Therefore, the results suggest stronger robustness across different lighting, road surfaces and damage conditions, although the values should still be treated as contextual comparisons because the datasets and experimental setups are different.

5.10 Discussion of the Research Questions

5.10.1 Research Question 1 - Effect of Combining Road Conditions

Combining daytime, nighttime and off-road imagery increased the diversity of Final 4, but it did not increase the overall mAP50. The score decreased from 0.824 in Final 2 to 0.764 in Final 4 as more varied data were added. This suggests that broader data improve coverage across different road conditions, but greater diversity also makes the detection task more difficult. Therefore, dataset quality and consistency are as important as dataset size.

5.10.2 Research Question 2 – One Combined Model or Two Specialist Models

The combined Final 5–Final 7 models performed reasonably for potholes but remained weak for cracks, with crack mAP50 between 0.333 and 0.465. In contrast, the dedicated crack models improved from 0.441 in Crack-only to 0.720 in Crack 3. Although the datasets and training settings also changed, the results support the final decision to use two separate specialist models rather than one combined detector.

5.10.3 Research Question 3 – Live Operation on Consumer Hardware

The final system successfully ran both Final 4 and Crack 3 on a live camera using the development laptop. Final 4 recorded 7.1 ms inference latency and Crack 3 recorded 8.9 ms on the RTX 5080 Laptop GPU, while the live detection also operated smoothly in practical testing. This confirms that the uncompressed system is suitable for real-time use on consumer hardware.

5.10.4 Compression and Deployment Finding

Although compression was framed in the project aim rather than as a separate stated research question, it produced an important deployment finding. FP16 ONNX export reduced the stored model size by only about 7%, but it preserved Final 4 mAP50 within 0.009 of the original and Crack 3 within 0.024. This makes FP16 the more defensible compression choice from the experiments carried out here. The result also shows why compression needs to be measured rather than assumed: reduced numerical precision did not automatically produce a dramatic file-size or speed benefit

5.11 Chapter Summary

The results confirm that the final system is best understood as two related but independent detectors. Final 4 achieved 0.764 validation mAP50 and performed particularly strongly on the dedicated night test subset, where mAP50 reached 0.895. The crack pipeline improved progressively from 0.441 mAP50 in Crack-only to 0.615 in Crack 2 and 0.720 in the final Crack 3 model. The confidence curves support the 0.35 deployment threshold, while the raw confusion matrices show that missed small or ambiguous defects and unmatched predictions remain the main practical errors.

The compression experiments showed that FP16 ONNX retained most of the original detection performance but produced only a modest file-size reduction, and a fair end-to-end speed comparison could not be completed. The qualitative application tests demonstrated image, video and live-camera operation together with the rule-based hazard score, but the risk value should be treated as an operational prioritisation heuristic rather than a measured engineering severity. These findings provide the basis for the conclusions and future-work recommendations in the following chapters.

CHAPTER 6: CONCLUSION

This project developed a practical dual-model road damage detection system using Final 4 for potholes and Crack 3 for cracks. The models were trained using varied road imagery covering daytime, nighttime, gravel/unpaved surfaces and different image qualities. Final 4 achieved an overall mAP50 of 0.764 and 0.895 on the dedicated night subset, while Crack 3 achieved an mAP50 of 0.720. The experiments also supported the use of two specialist detectors, since the combined models repeatedly showed weaker crack performance.

The completed system goes beyond detection by adding a 1-10 rule-based risk score and a Flask application that processes images, videos and live camera input. The original PyTorch models also operated smoothly in live testing on the development laptop. Compression experiments showed that FP16 ONNX retained most of the detection performance, but produced only a modest reduction in model size, while INT8 evaluation could not be completed reliably in the current setup.

Overall, the project demonstrates that separate pothole and crack models can provide useful multi-condition road damage detection on consumer hardware. At the same time, the results identify clear areas for further improvement in compression, output integration, model accuracy and validation of the automated risk score.

CHAPTER 7: FUTURE WORK

7.1 Quantisation and Deployment Optimisation

Future work should revisit INT8 quantisation using improved calibration, quantisation-aware training or a GPU-compatible deployment backend such as TensorRT. The original and compressed models should then be benchmarked under the same hardware and execution backend so that changes in FPS, model size and mAP can be compared fairly.

7.2 Unified Detection Output

The current application displays separate pothole and crack outputs. A future version could keep the two specialist models internally but combine their detections into one annotated image, video or live-camera view, with different labels and colours for each defect type and a single risk summary.

7.3 Model Accuracy and Attention Mechanisms

Further experiments could modify or compare attention mechanisms, backbone and feature-fusion components to improve detection accuracy, especially for small or difficult cracks. Any architectural change should be tested across the same varied daytime, nighttime and road-surface conditions rather than on one narrow dataset.

7.4 Data and Risk-Score Validation

More genuine nighttime crack images, adverse-weather scenes, glare and additional unpaved-road data could improve generalisation. The 1-10 risk score could also be validated against physical measurements such as pothole depth, defect size and crack width so that future versions can provide a more meaningful engineering severity estimate.

REFERENCES

1. Abdelwahed, S.H. *et al.* (2025) “Advancements in real-time road damage detection: a comprehensive survey of methodologies and datasets,” 22, p. 137. Available at: https://doi.org/10.1007/s11554-025-01683-1.
2. Alkalbani, A.M. *et al.* (2026) “RDD4D: 4D-Attention-Guided Road Damage Detection and Classification,” *IEEE Access*, 14, pp. 40078–40092. Available at: https://doi.org/10.1109/ACCESS.2026.3666237.
3. Arya, D. *et al.* (2024) “RDD2022: A multi-national image dataset for automatic road damage detection,” *Geoscience Data Journal*, 11(4), pp. 846–862. Available at: https://doi.org/10.1002/GDJ3.260.
4. Bai, J. and Zhang, S. (2025) “RF-YOLO-RASFF: An End-to-End Framework for Nighttime Road Defect Detection,” *2025 8th International Conference on Computer Network, Electronic and Automation, ICCNEA 2025*. Institute of Electrical and Electronics Engineers Inc. Available at: https://doi.org/10.1109/ICCNEA66167.2025.11211868.
5. Bučko, B ; *et al.* (2022) “Computer Vision Based Pothole Detection under Challenging Conditions,” *Sensors 2022, Vol. 22, Page 8878*, 22(22), p. 8878. Available at: https://doi.org/10.3390/S22228878.
6. Du, Y. *et al.* (2024) “A Set of Effective Strategies for Optimized Road Damage Detection,” *Proceedings - 2024 IEEE International Conference on Big Data, BigData 2024*. Institute of Electrical and Electronics Engineers Inc., pp. 8447–8452. Available at: https://doi.org/10.1109/BIGDATA62323.2024.10825491.
7. Hu, Y. *et al.* (2025) “Advanced low-light image transformation for accurate nighttime pavement distress detection,” *Computer-Aided Civil and Infrastructure Engineering*, 40(28), pp. 5307–5324. Available at: https://doi.org/10.1111/mice.70001.
8. Khan, M. *et al.* (2023) “Pothole detection for autonomous vehicles using deep learning: a robust and efficient solution,” *Frontiers in Built Environment*, 9, p. 1323792. Available at: https://doi.org/10.3389/FBUIL.2023.1323792/TEXT.
9. Khan, M.W. *et al.* (2024) “Real-Time Road Damage Detection and Infrastructure Evaluation Leveraging Unmanned Aerial Vehicles and Tiny Machine Learning,” *IEEE Internet of Things Journal*, 11(12), pp. 21347–21358. Available at: https://doi.org/10.1109/JIOT.2024.3385994.
10. Khare, O. *et al.* (2023) “YOLOv8-Based Visual Detection of Road Hazards: Potholes, Sewer Covers, and Manholes,” *2023 IEEE Pune Section International Conference, PuneCon 2023*. Institute of Electrical and Electronics Engineers Inc. Available at: https://doi.org/10.1109/PUNECON58714.2023.10449999.
11. Lin, L., Zhi, X. and Zhang, Z. (2023) “A Lightweight Method for Automatic Road Damage Detection Based on Deep Learning,” *2023 3rd International Conference on Electronic Information Engineering and Computer Science, EIECS 2023*. Institute of Electrical and Electronics Engineers Inc., pp. 81–85. Available at: https://doi.org/10.1109/EIECS59936.2023.10435623.
12. Ling, M. *et al.* (2024) “Nighttime Pothole Detection: A Benchmark,” *Electronics 2024, Vol. 13, Page 3790*, 13(19), p. 3790. Available at: https://doi.org/10.3390/ELECTRONICS13193790.
13. Pham, V., Nguyen, D. and Donan, C. (2022) “Road Damage Detection and Classification with YOLOv7,” *Proceedings - 2022 IEEE International Conference on Big Data, Big Data 2022*. Institute of Electrical and Electronics Engineers Inc., pp. 6416–6423. Available at: https://doi.org/10.1109/BIGDATA55660.2022.10020856.
14. Priyanka, M., Praveen Raja, P. and Akhib, M. (2023) “Smartphone Image-Based Road Damage Detection in Low-Light Conditions,” *2023 International Conference on Network, Multimedia and Information Technology, NMITCON 2023*. Institute of Electrical and Electronics Engineers Inc. Available at: https://doi.org/10.1109/NMITCON58196.2023.10275937.
15. Tsanaullailla, L. *et al.* (2024) “Road Pothole Damage Detector at Night Using YOLOv8,” *ICAISD 2024 - International Conference on Advanced Information Scientific Development: AI for Investing the Sustainability Development of Human Living Digitally, Proceedings*. Institute of Electrical and Electronics Engineers Inc., pp. 138–143. Available at: https://doi.org/10.1109/ICAISD63055.2024.10894890.
16. Verma, P. *et al.* (2025) “HPC-enabled Deep Learning Multi-model for Road Damage Detection,” *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences*. Copernicus GmbH, pp. 701–706. Available at: https://doi.org/10.5194/ISPRS-ANNALS-X-5-W2-2025-701-2025.
17. Wan, F. *et al.* (2022) “YOLO-LRDD: a lightweight method for road damage detection based on improved YOLOv5s,” *EURASIP Journal on Advances in Signal Processing 2022 2022:1*, 2022(1), pp. 98-. Available at: https://doi.org/10.1186/S13634-022-00931-X.

18. Wang, W. *et al.* (2025) “YOLO-RD: A Road Damage Detection Method for Effective Pavement Maintenance,” *Sensors 2025, Vol. 25, Page 1442*, 25(5), p. 1442. Available at: https://doi.org/10.3390/S25051442.
19. Xu, X. *et al.* (2025) “Enhanced pothole detection in low-light conditions using DCE-Pothole and YOLOv10 fusion,” *Results in Engineering*, 28, p. 107322. Available at: https://doi.org/10.1016/J.RINENG.2025.107322.
20. Zanevych, Y. *et al.* (2024) “Evaluation of Pothole Detection Performance Using Deep Learning Models Under Low-Light Conditions,” *Sustainability 2024, Vol. 16, Page 10964*, 16(24), p. 10964. Available at: https://doi.org/10.3390/SU162410964.
21. Zhang, Y. *et al.* (2022) “Automatic Recognition of Road Damage Based on Lightweight Attentional Convolutional Neural Network,” *Sensors 2022, Vol. 22, Page 9599*, 22(24), p. 9599. Available at: https://doi.org/10.3390/S22249599.